Comp 4911 Process Review

Copyright © 1998 Rational Software, all rights reserved

R

Best Practices of Software Engineering

Copyright © 1998 Rational Software, all rights reserved

R

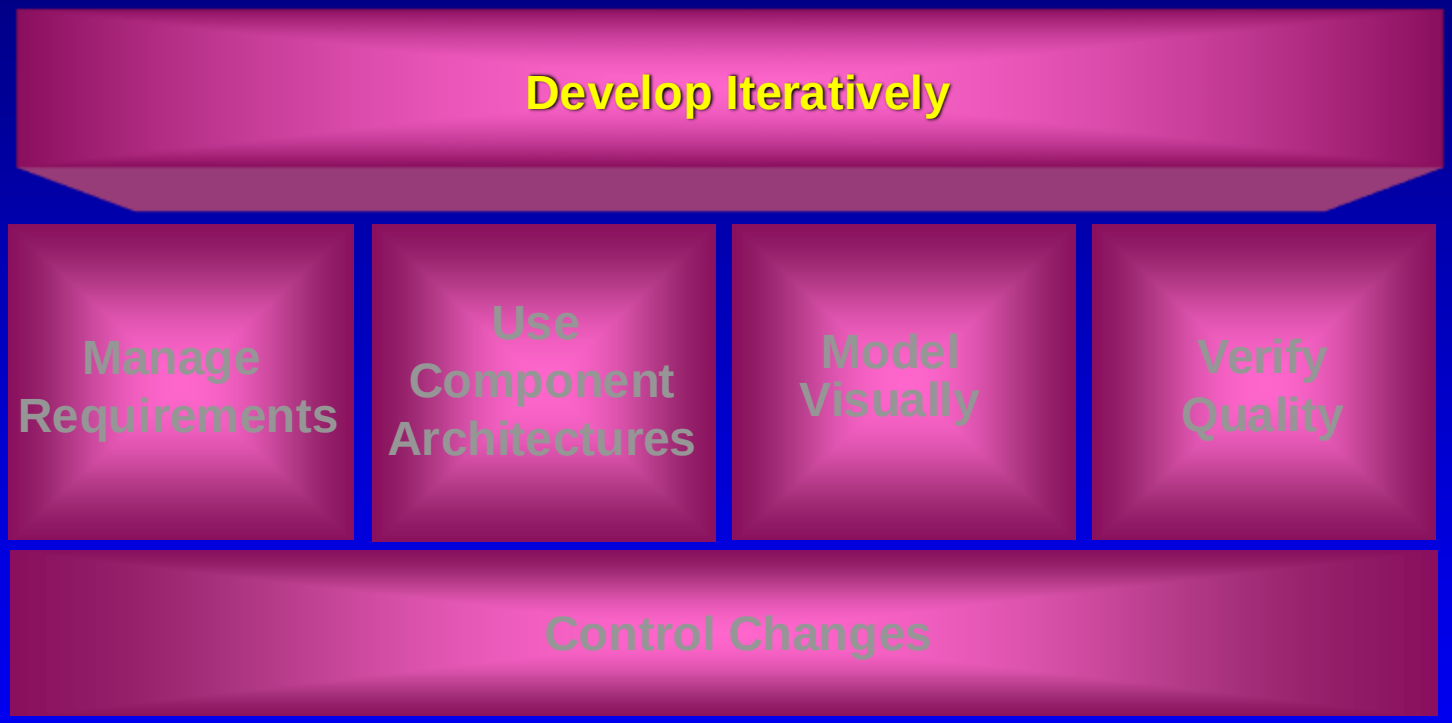
Objectives: Best Practices of Software Engineering

- Symptoms and root causes of software development problems
- Best practices for software development
- Look at how best practices address the root causes of software development problems

Copyright © 1998 Rational Software, all rights reserved

R

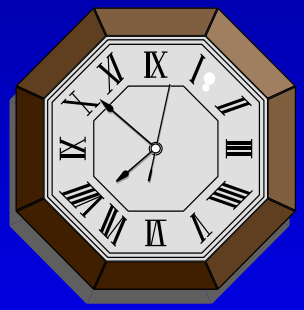
Practice 1: Develop Software Iteratively



Copyright © 1998 Rational Software, all rights reserved

Practice 1: Develop Software Iteratively

- An initial design will likely be flawed with respect to its key requirements
- Late-phase discovery of design defects results in costly over-runs and/or project cancellation

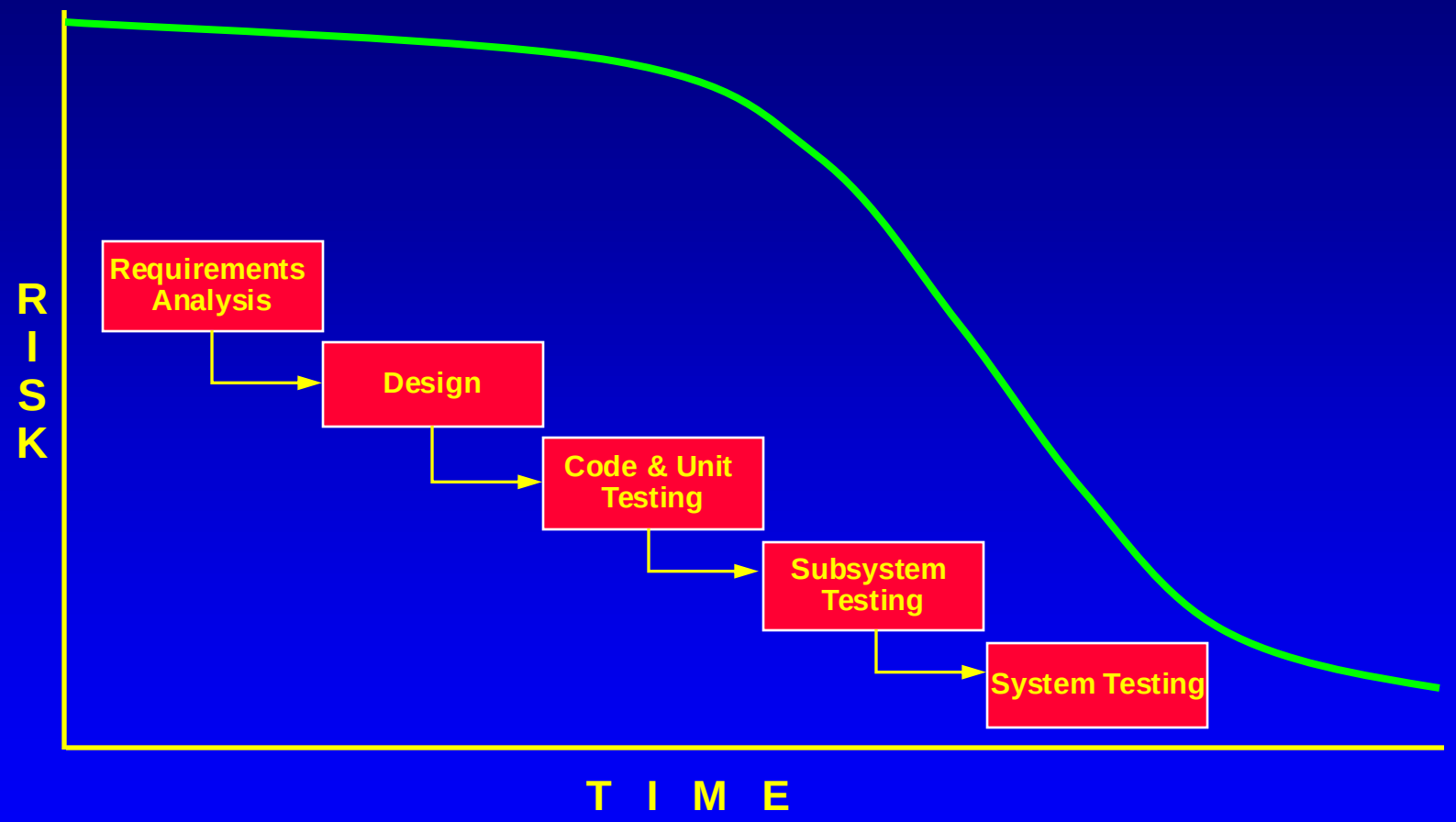


The time and money spent implementing a faulty design are not recoverable

Copyright © 1998 Rational Software, all rights reserved

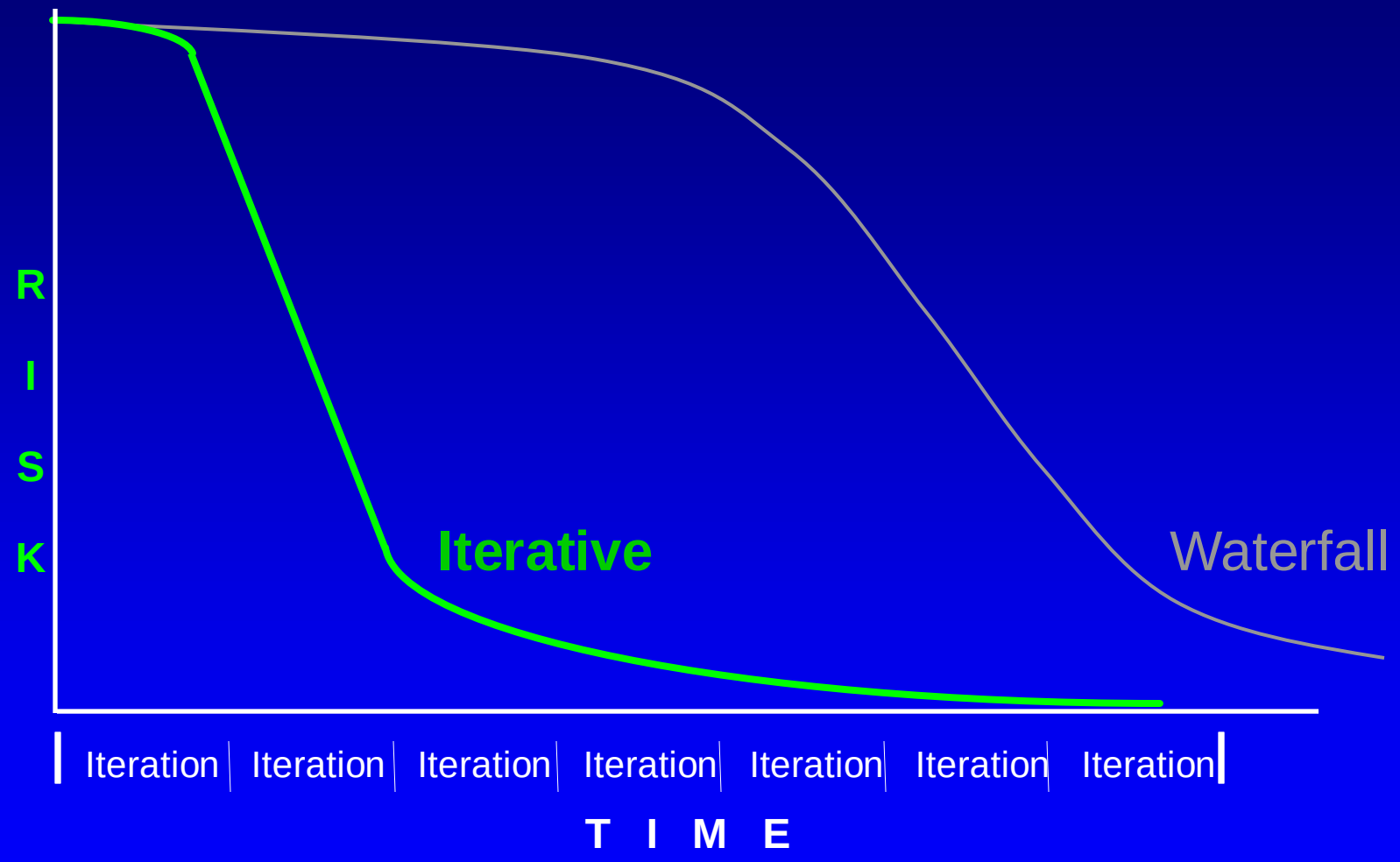
R

Waterfall Development Delays Reduction of Risk



Copyright © 1998 Rational Software, all rights reserved

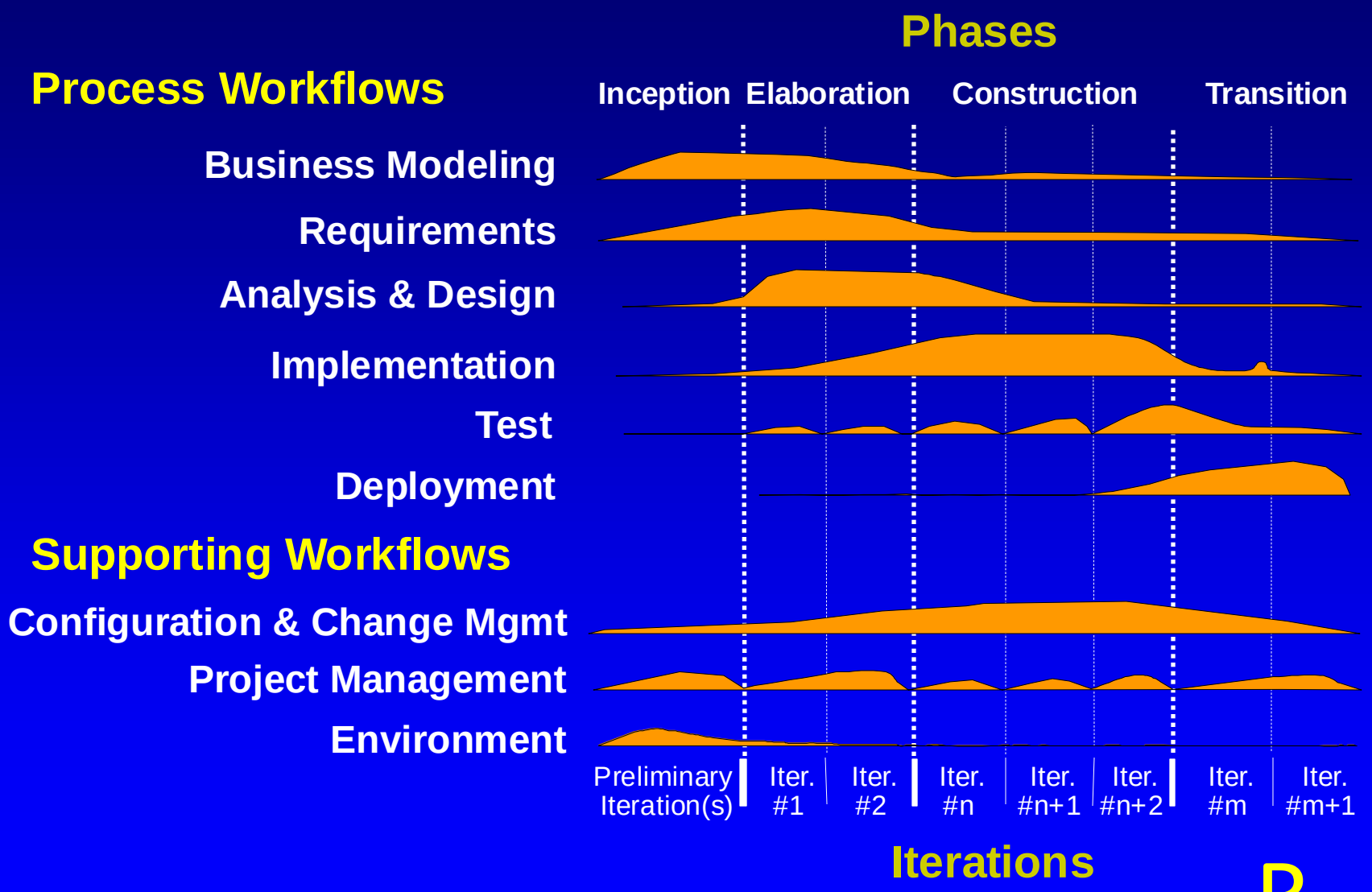
Iterative Development Accelerates Risk Reduction



Copyright © 1998 Rational Software, all rights reserved

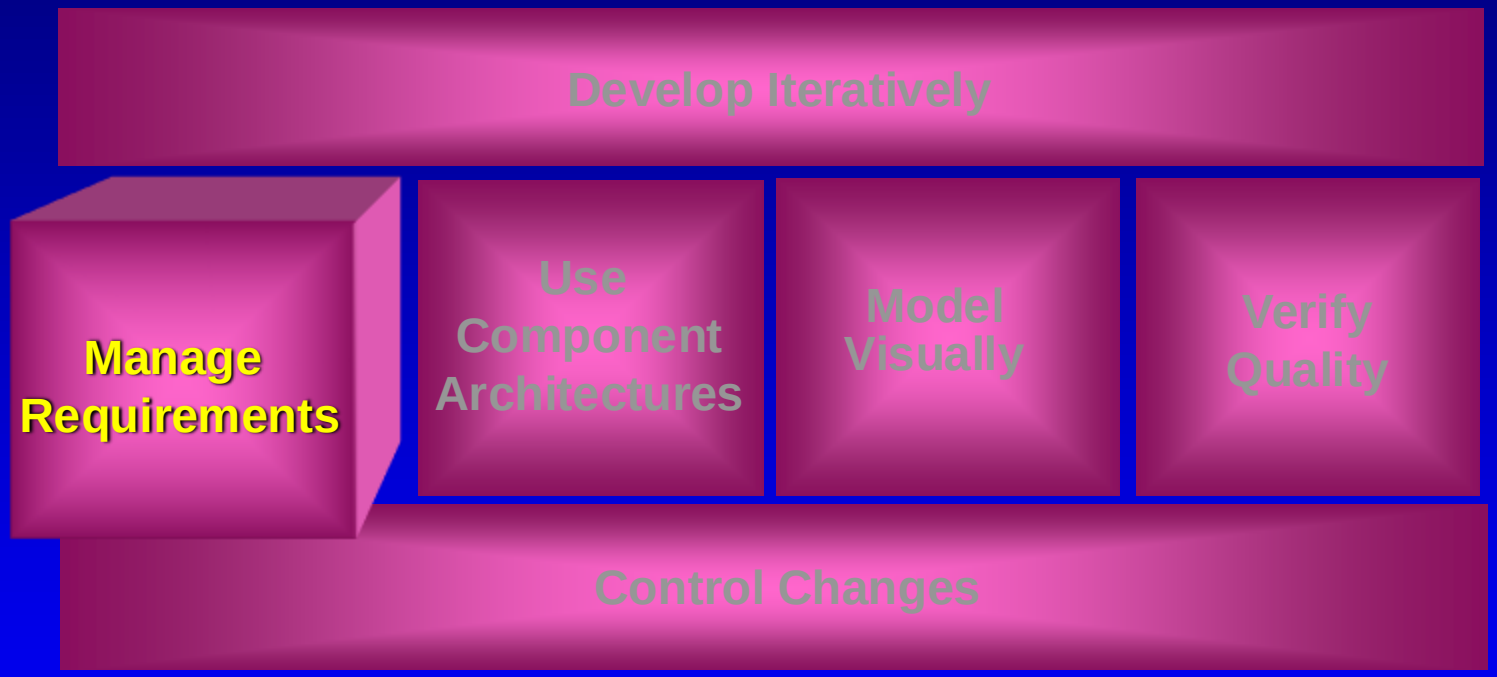
R

Apply Best Practices Throughout the Life Cycle



Copyright © 1998 Rational Software, all rights reserved

Practice 2: Manage Requirements

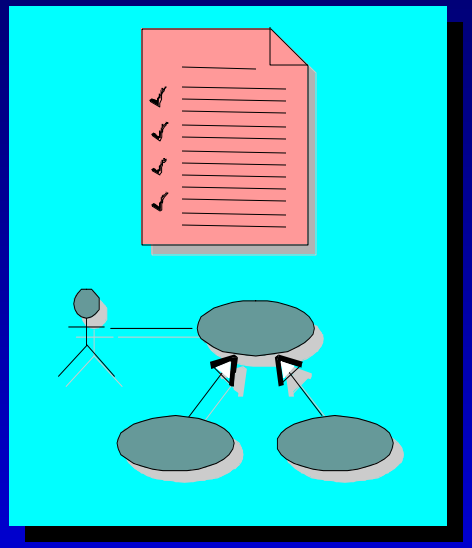


Copyright © 1998 Rational Software, all rights reserved

R

Practice 2: Manage Requirements

- Elicit, organize, and document required functionality and constraints
- Evaluate changes and determine their impact
- Track and document tradeoffs and decisions

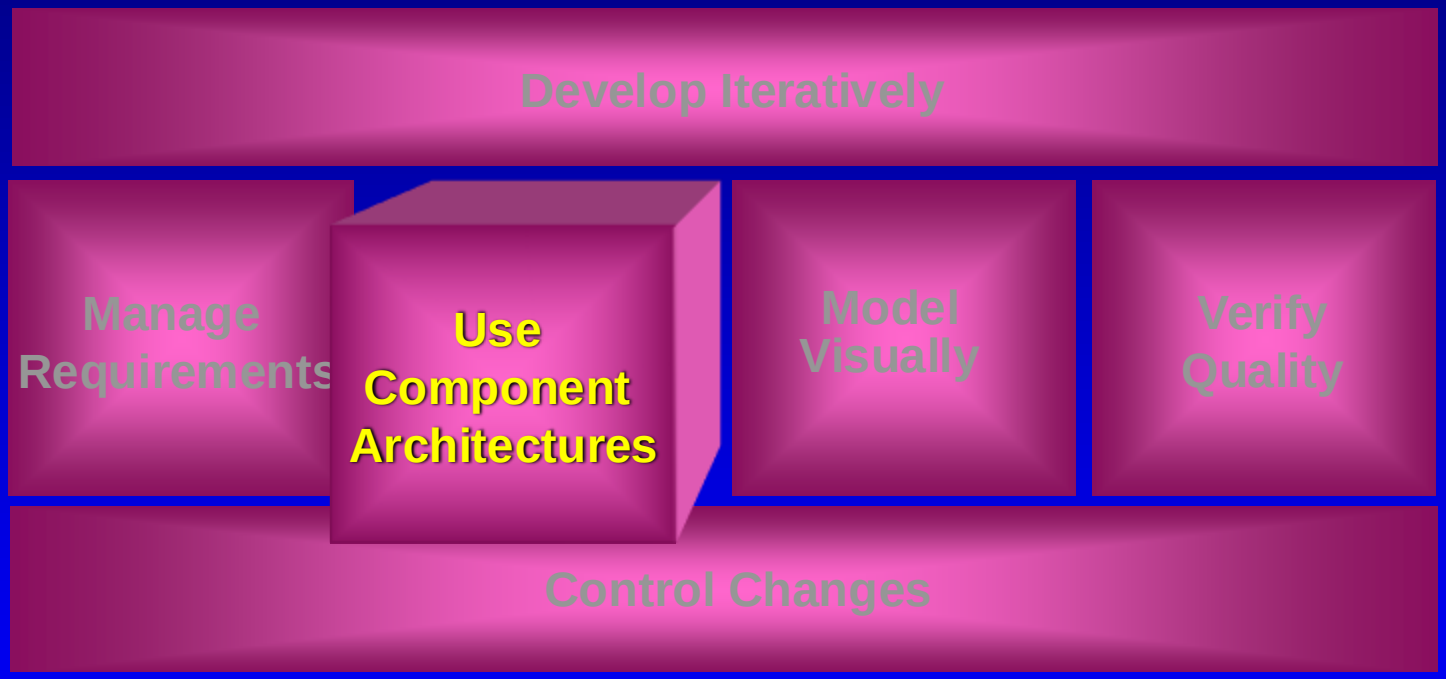


Requirements are dynamic -- Expect them to change during software development

R

Copyright © 1998 Rational Software, all rights reserved

Practice 3: Use Component-Based Architectures



Copyright © 1998 Rational Software, all rights reserved

Software Architecture Defined

- **Software architecture** encompasses significant decisions about the organization of a software system
  - Selection of the structural elements and their interfaces by which a system is composed
  - Behavior as specified in collaborations among those elements
  - Composition of these structural and behavioral elements into progressively larger subsystems
  - Architectural style that guides this organization, these elements and their interfaces, their collaborations, and their composition

Copyright © 1998 Rational Software, all rights reserved

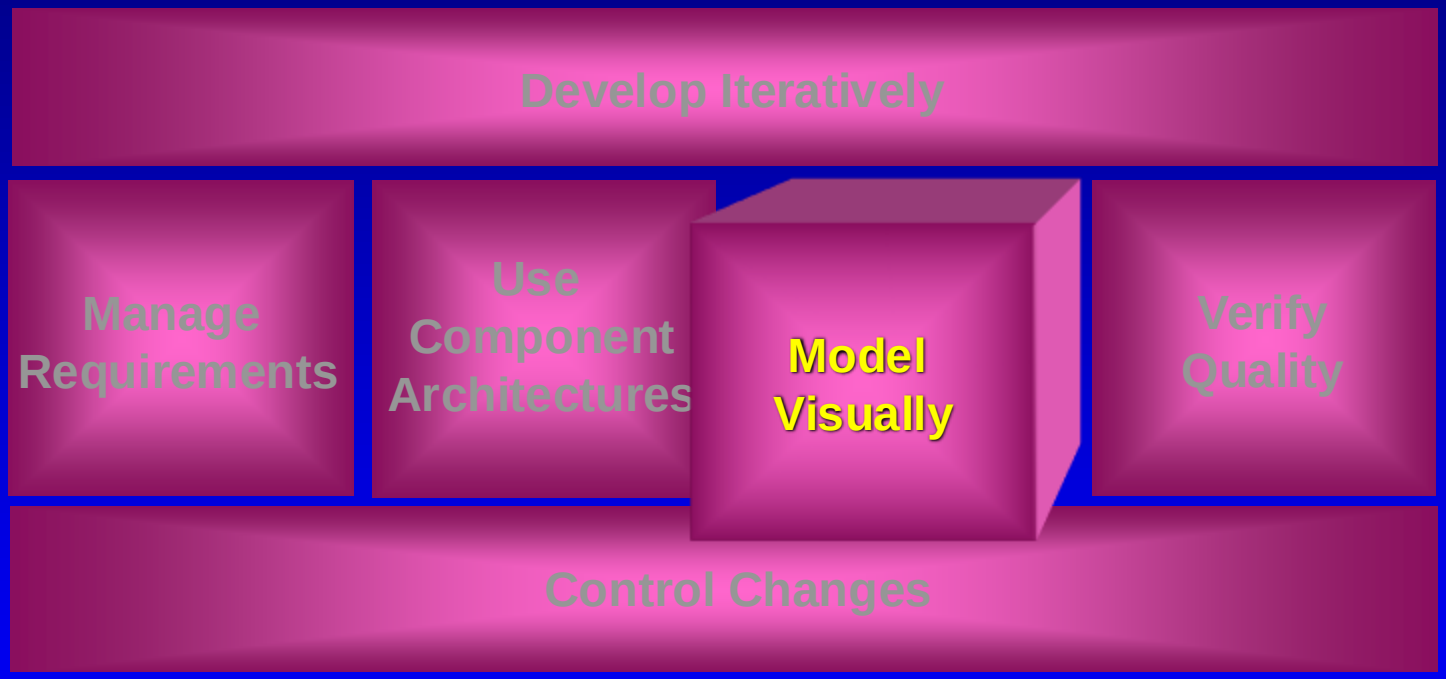
R

Architectural Concerns

- Software architecture is concerned with structure and behavior and context:
  - Usage
  - Functionality
  - Performance
  - Resilience
  - Reuse
  - Comprehensibility
  - Economic and technological constraints and tradeoffs
  - Aesthetics

Copyright © 1998 Rational Software, all rights reserved

Practice 4: Visually Model Software



Copyright © 1998 Rational Software, all rights reserved

Practice 4: Visually Model Software

- Capture the structure and behavior of architectures and components
- Show how the elements of the system fit together
- Hide or expose details as appropriate for the task
- Maintain consistency between a design and its implementation
- Promote unambiguous communication

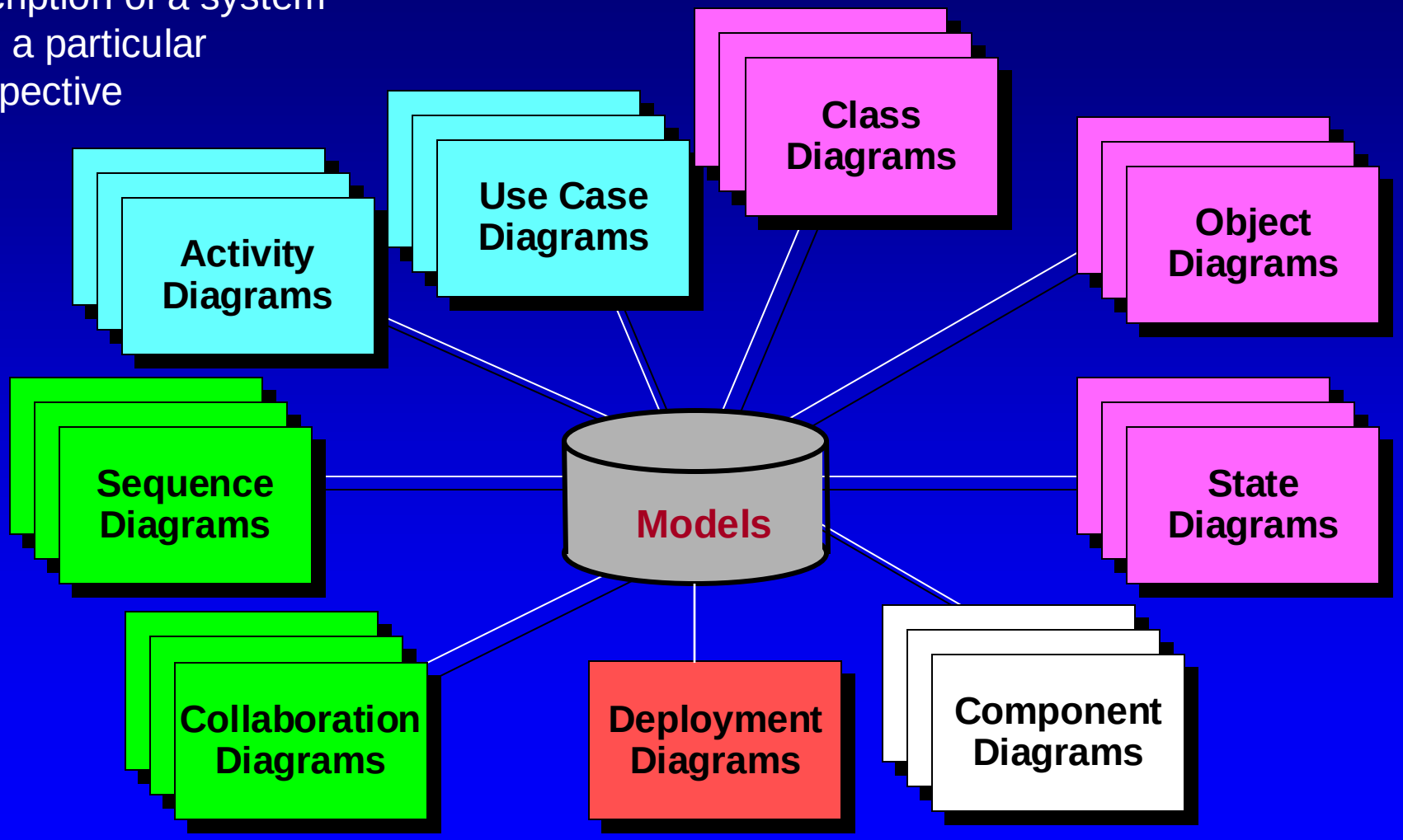
Visual modeling improves our ability to manage software complexity

Copyright © 1998 Rational Software, all rights reserved

R

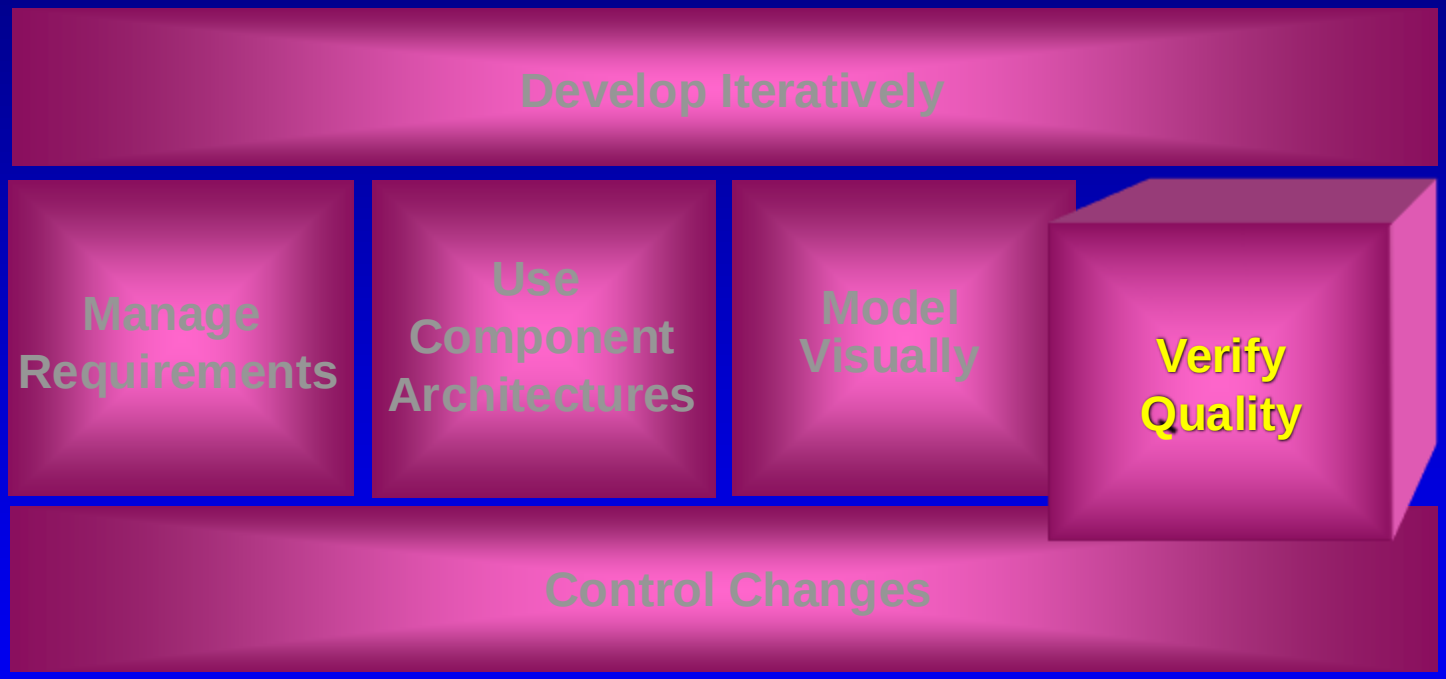
Diagrams Are Views of a Model

A *model* is a complete description of a system from a particular perspective



Copyright © 1998 Rational Software, all rights reserved

Practice 5: Verify Software Quality

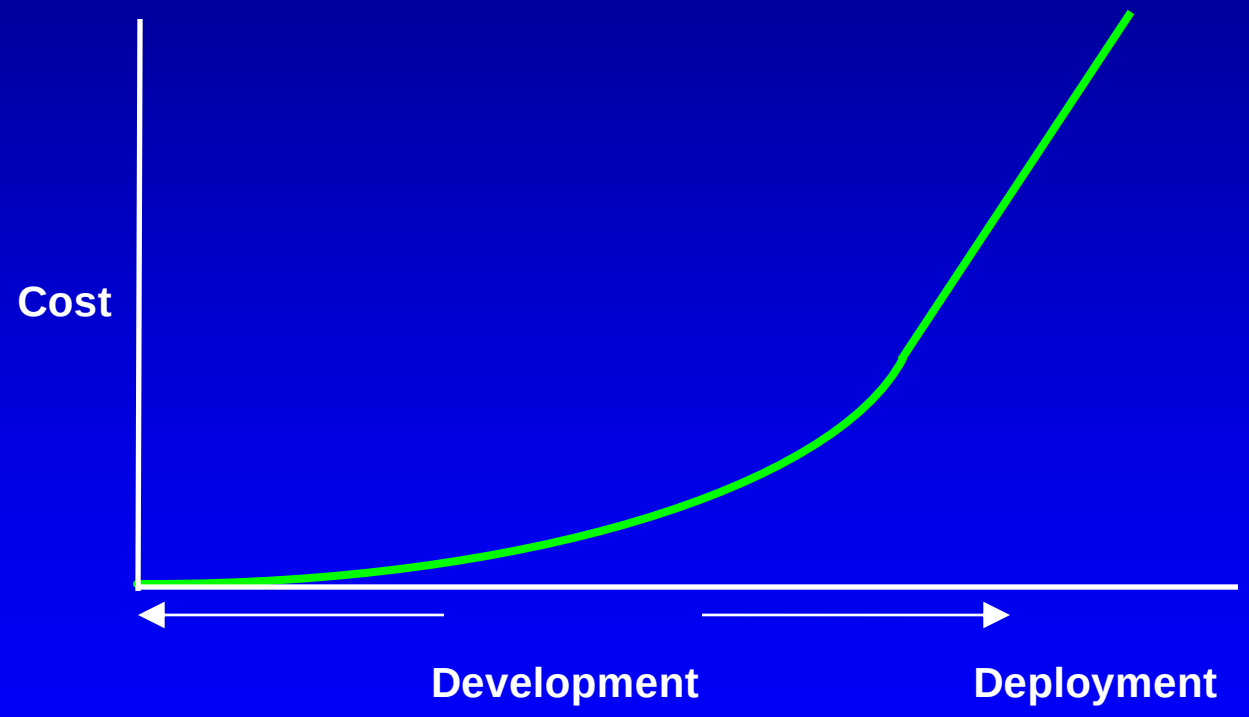


Copyright © 1998 Rational Software, all rights reserved

R

Practice 5: Verify Software Quality

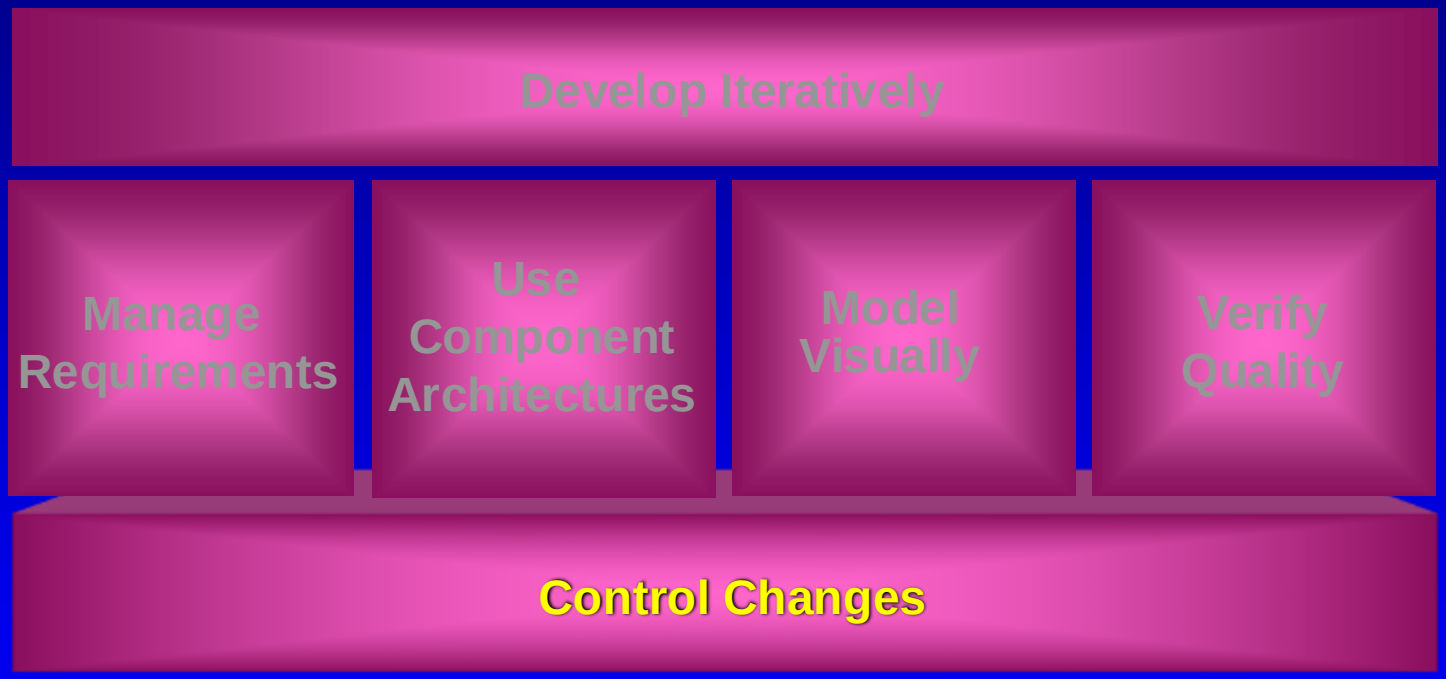
Software problems are 100 to 1000 times more costly to find and repair after deployment



Copyright © 1998 Rational Software, all rights reserved

R

Practice 6: Control Changes to Software



Copyright © 1998 Rational Software, all rights reserved

Introduction to Rational Unified Process

R

Copyright © 1998 Rational Software, all rights reserved

Objectives: Rational Unified Process

- Describe the **Unified Modeling Language (UML)**
- Define what a **software development process** is
- Describe the **Rational Unified Process**
- Explain the four **phases** of the Rational Unified Process and their associated milestones
- Define **iterations** and their relation to phases
- Explain the relations between:
  - Models and workflows
  - Phases, iterations, and workflows
- Define **artifact**, **worker**, and **activity**
- State the importance of automated tool support

R

Copyright © 1998 Rational Software, all rights reserved

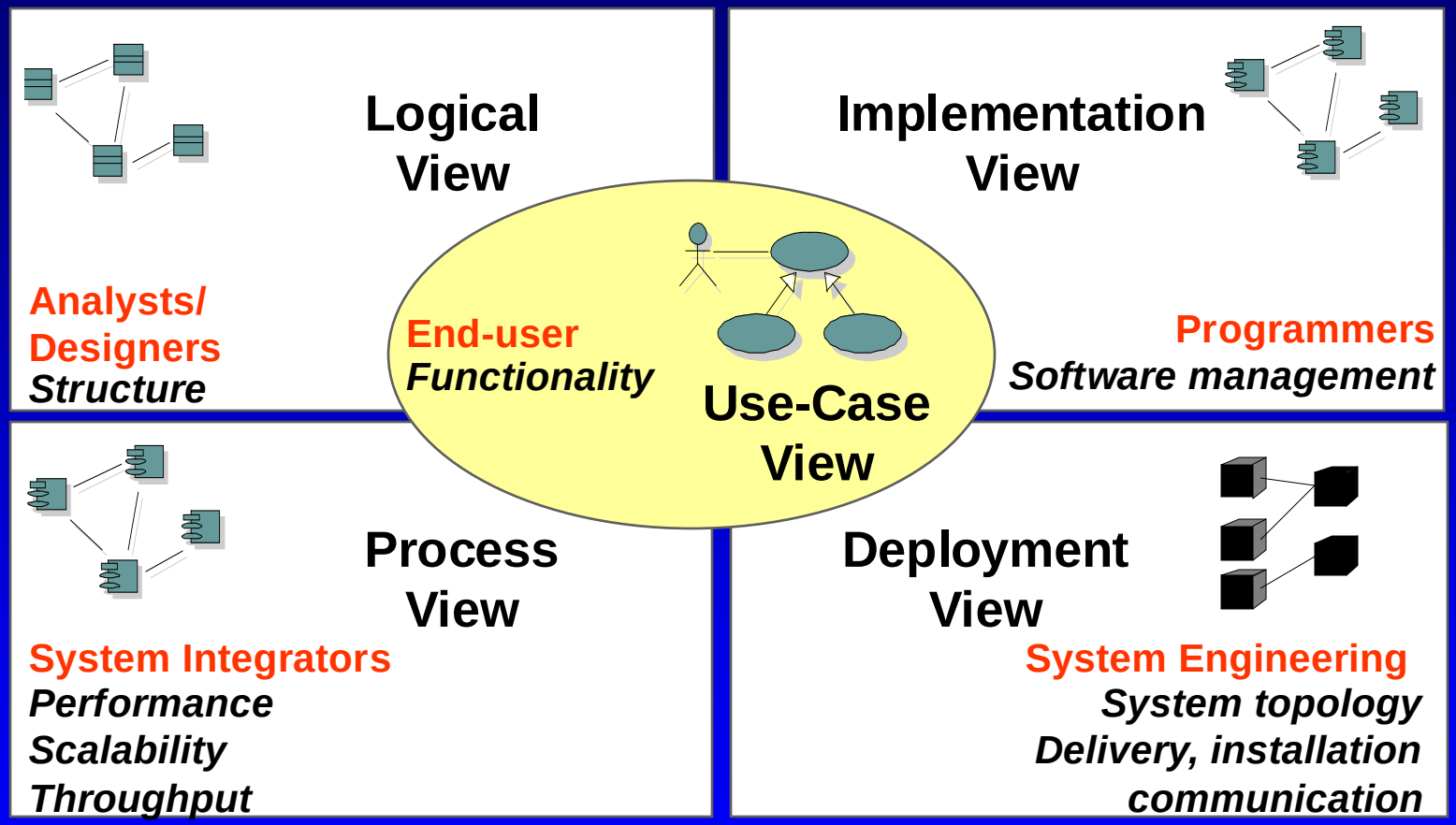
Rational Unified Process Is Architecture-Centric

- Architecture is the focus of the early iterations
  - Building, validating, and baselining the architecture constitute the primary objective of elaboration
- The Architectural Prototype validates the architecture and serves as the baseline for the rest of development
- The Software Architecture Document is the primary artifact that describes the architecture chosen
- Other artifacts derive from architecture:
  - Design guidelines including use of patterns and idioms
  - Product structure
  - Team structure

Copyright © 1998 Rational Software, all rights reserved

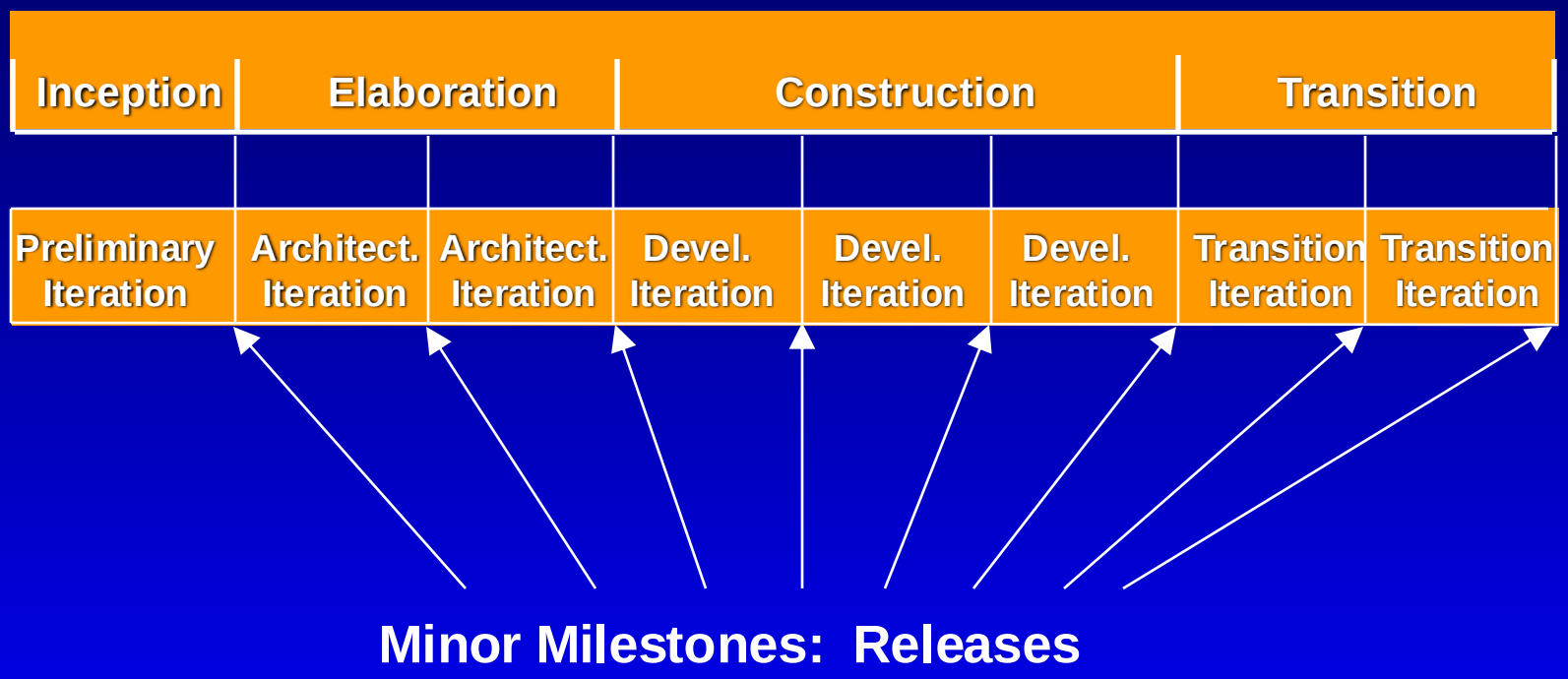
R

Representing Architecture: The 4+1 View Model



Copyright © 1998 Rational Software, all rights reserved

Iterations and Phases

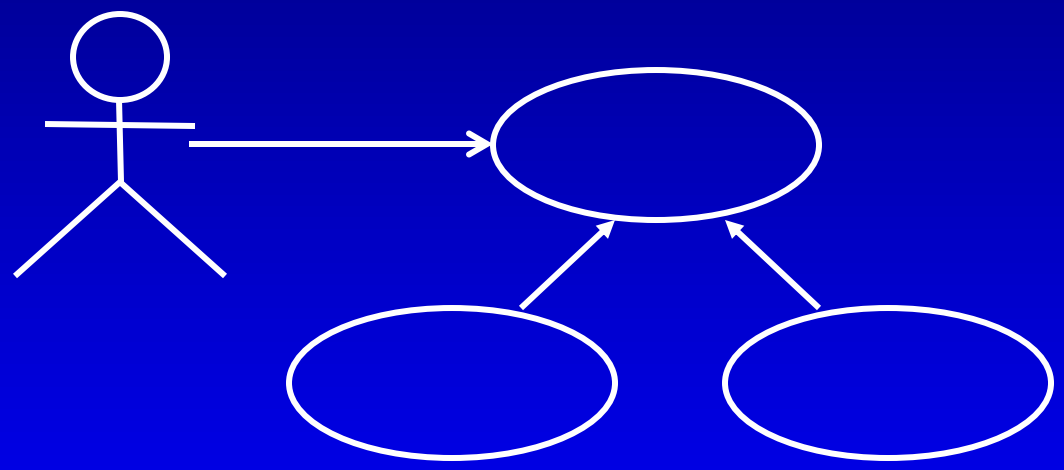


An iteration is a distinct sequence of activities with an established plan and evaluation criteria, resulting in an executable release (internal or external)

R

Copyright © 1998 Rational Software, all rights reserved

Use-Case-Driven Development

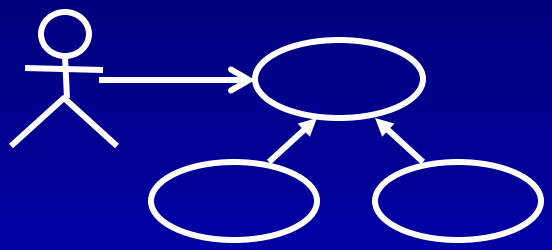


Use-Case Model

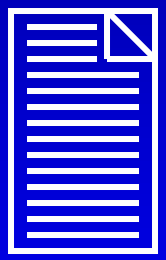
Copyright © 1998 Rational Software, all rights reserved

R

Use Cases Drive Analysis & Design

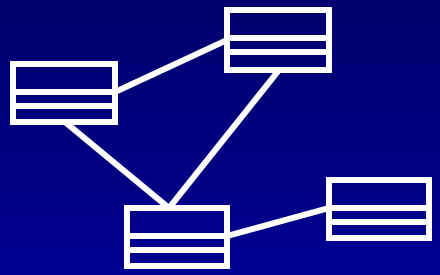


Use-Case Model

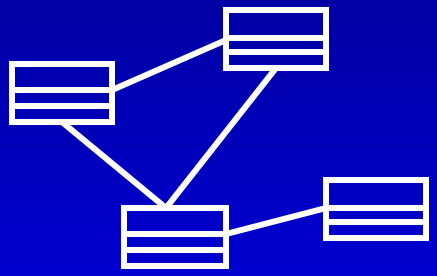


Supplementary Specifications

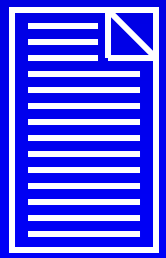
Analysis and Design



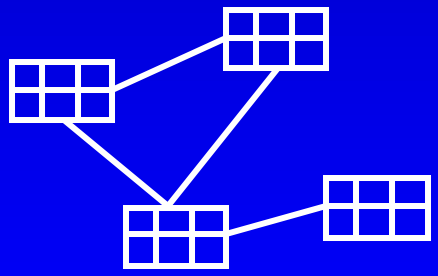
Analysis Model (optional)



Design Model



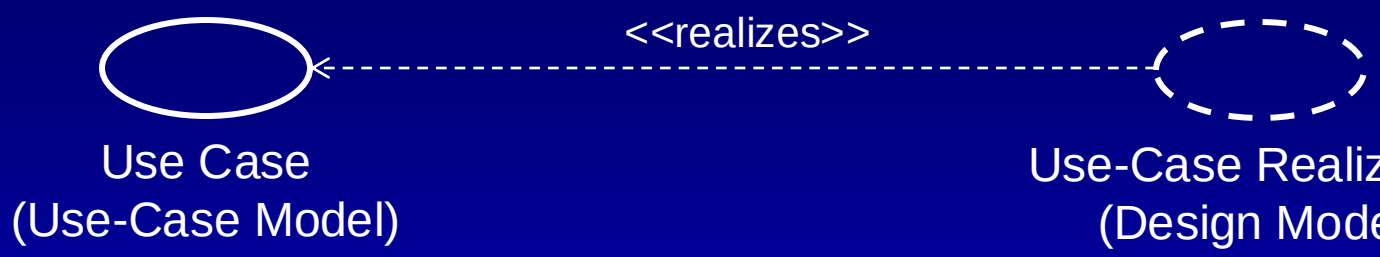
Architecture Document

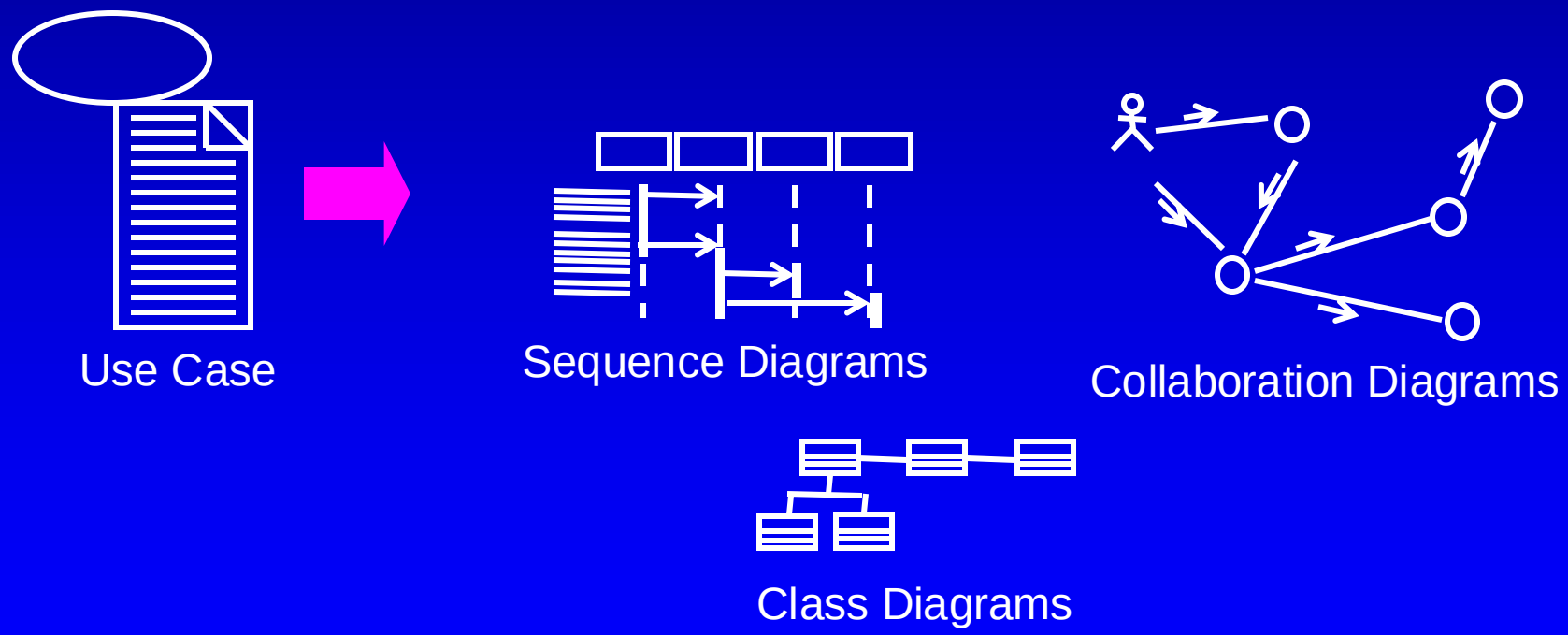


Data Model

Copyright © 1998 Rational Software, all rights reserved

Use Case Realization in Analysis & Design

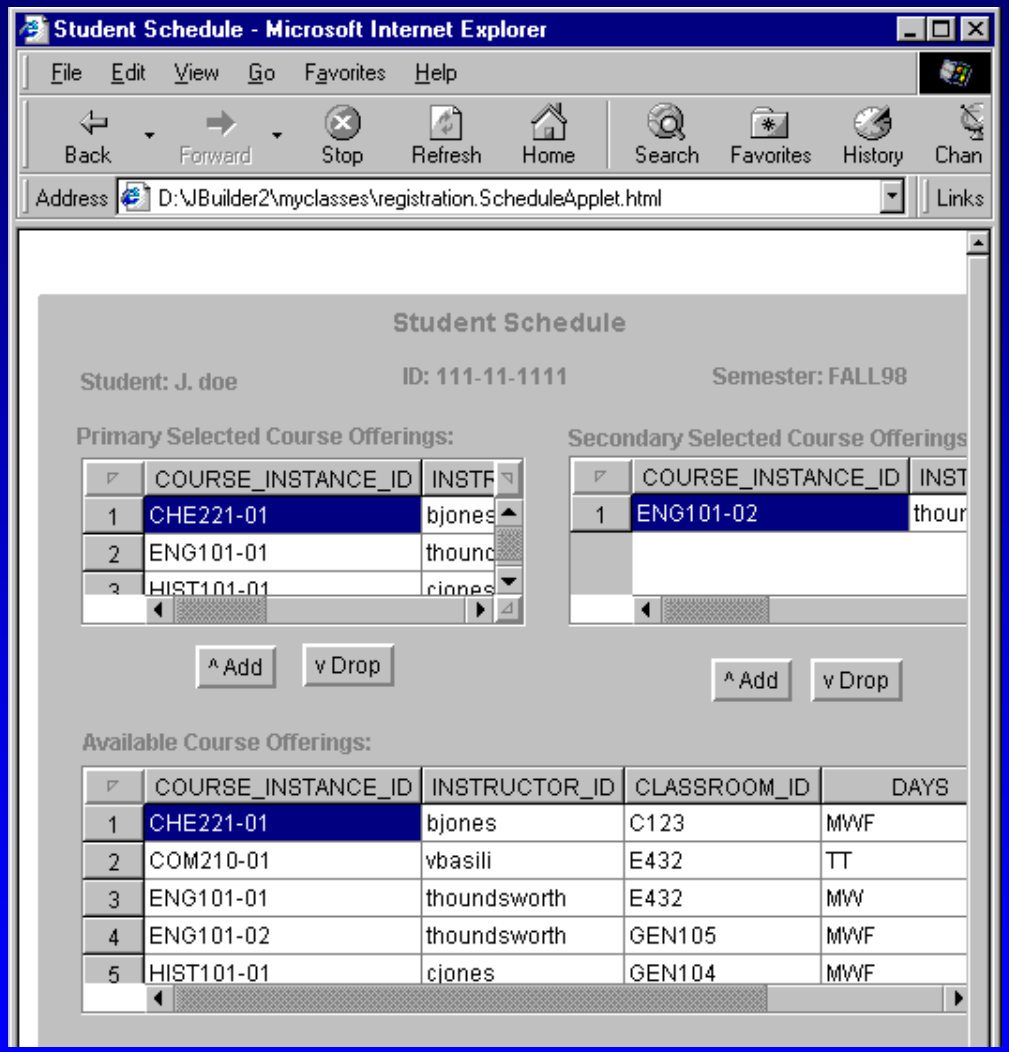




Copyright © 1998 Rational Software, all rights reserved

Use Cases Help in Defining User Interfaces

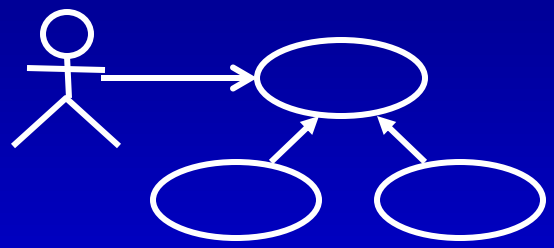
- Sketches of the system's proposed screen appearance can be derived from use cases
- These form the basis of user interface design



R

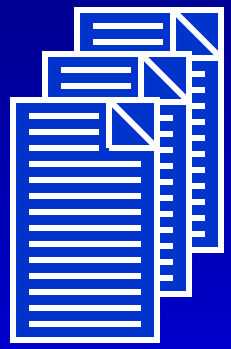
Copyright © 1998 Rational Software, all rights reserved

Use Cases and End-User Documentation



Use-Case Model

Deployment



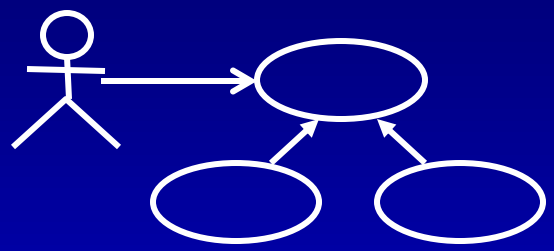
<u>End User Support Material</u>

- User's Guide
- On-line Help
- Demos
- Tutorials
- Training Material

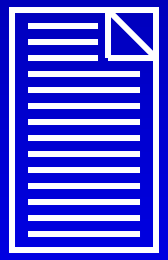
Copyright © 1998 Rational Software, all rights reserved

R

Use Cases Drive the Test Model

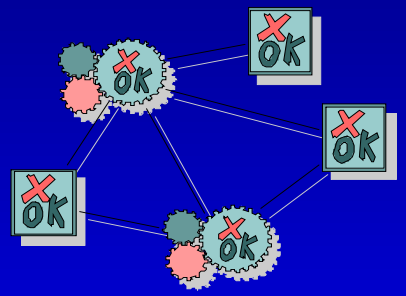


Use-Case Model



Supplementary Specifications

Test

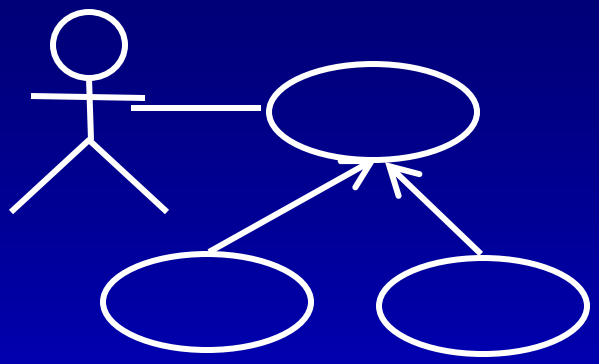


Test Model

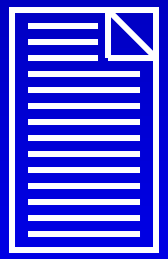
Copyright © 1998 Rational Software, all rights reserved

R

Use Cases Help in Iteration Planning



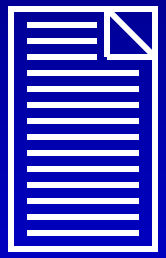
Use-Case Model



Supplementary Specification

Constraints

Project Management



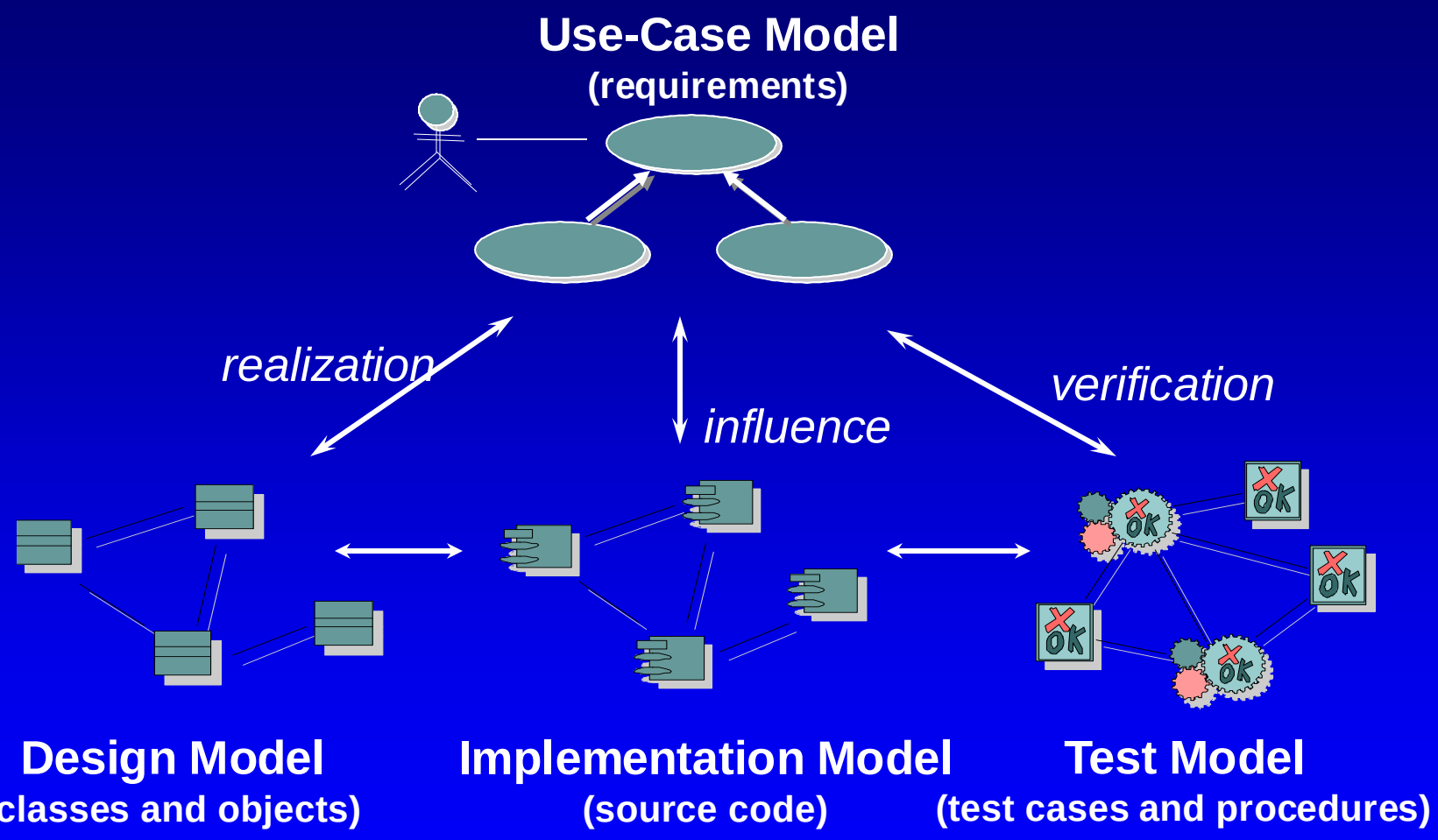
Iteration Plan

Fine-grained plan for a single iteration

Copyright © 1998 Rational Software, all rights reserved

R

The Use Case Model Relates to Other Models



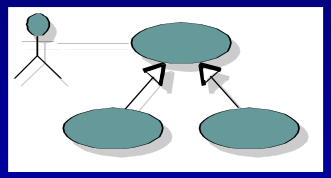
Copyright © 1998 Rational Software, all rights reserved

Architecture-Centric Development

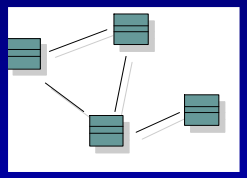
Copyright © 1998 Rational Software, all rights reserved

R

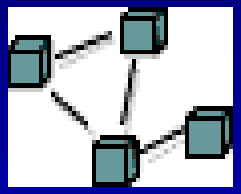
Architecture Is Represented by Views



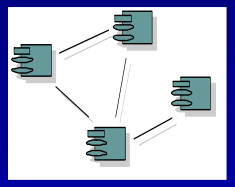
Use-Case Model



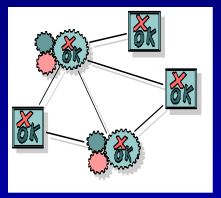
Design Model



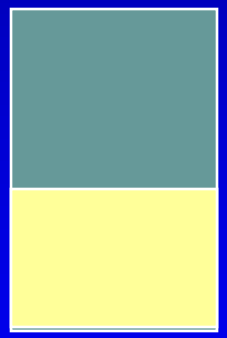
Deployment Model

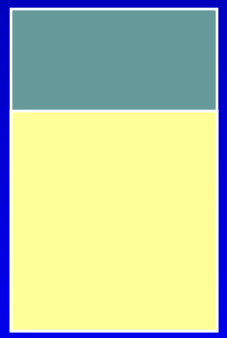


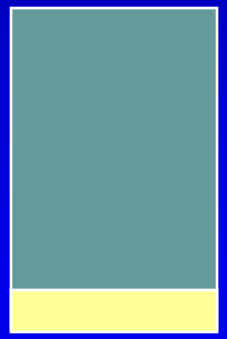
Implementation Model



Test Model





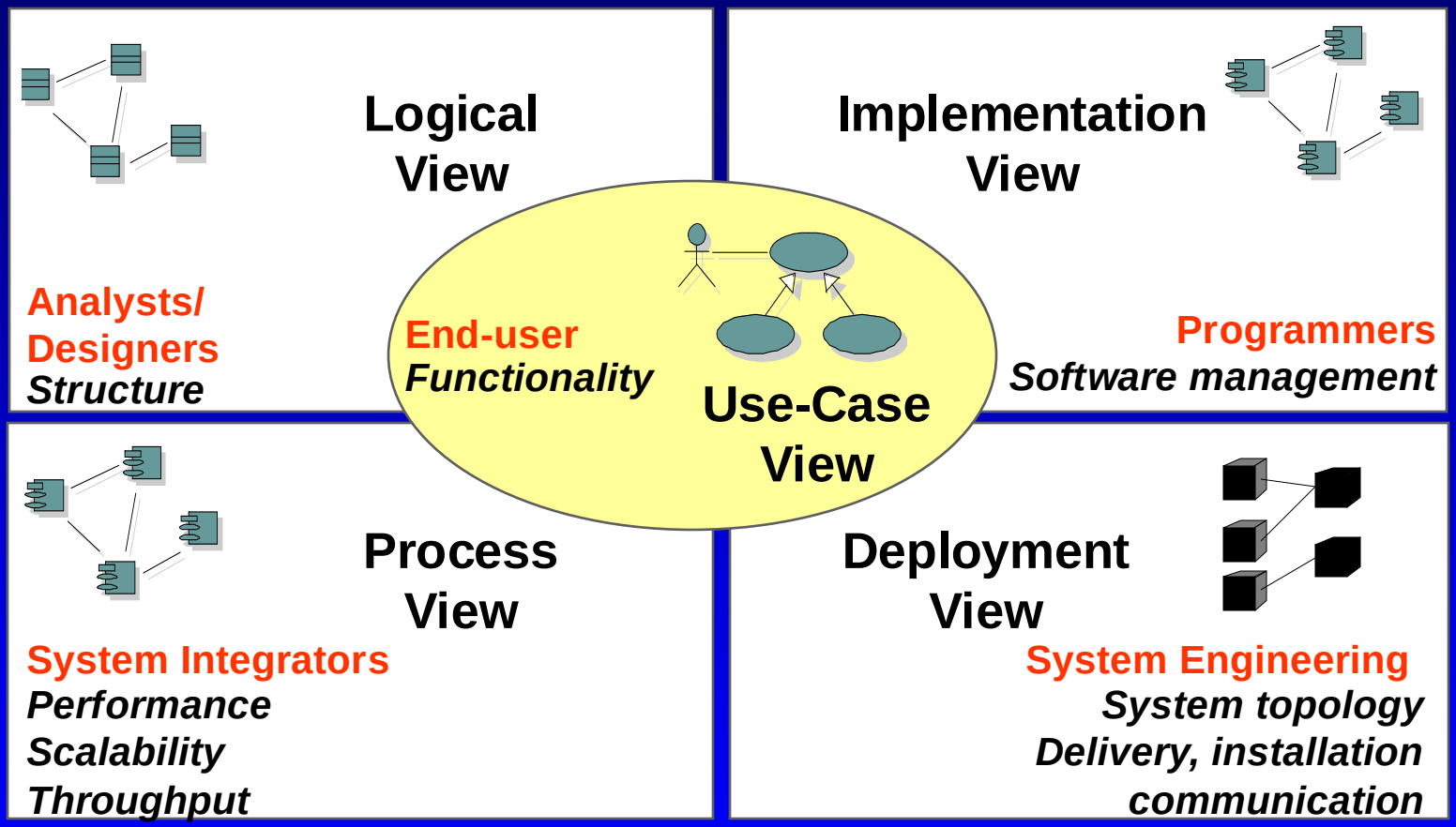


Use Case View

Implementation View

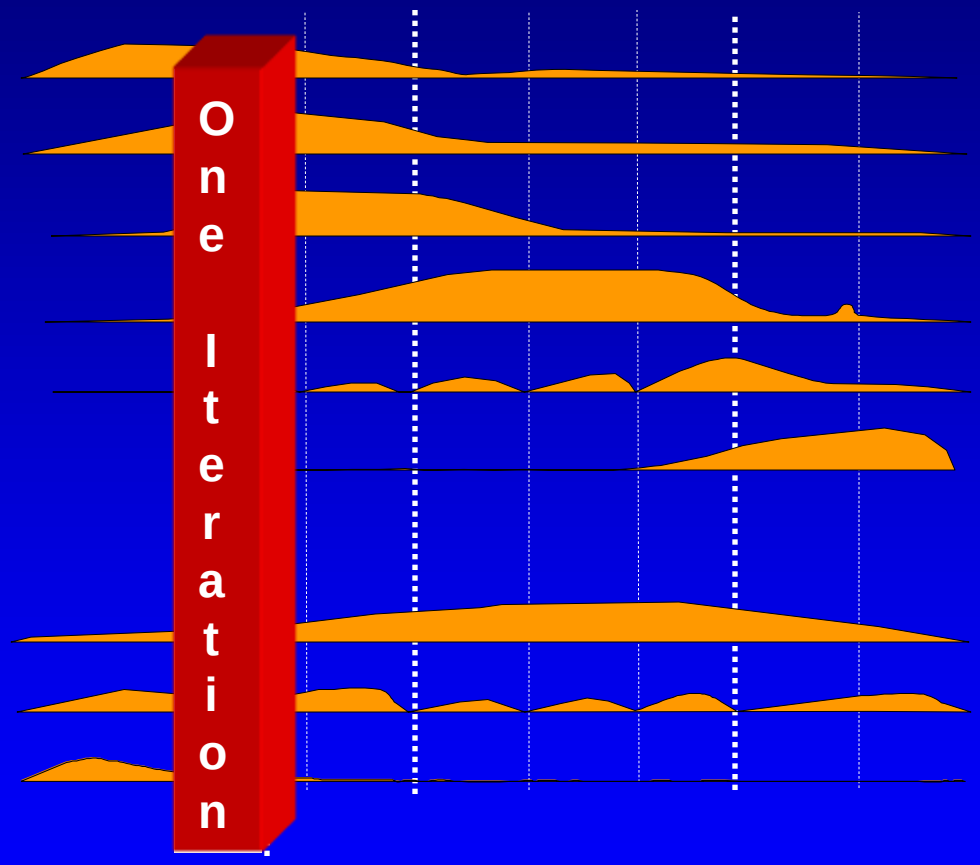
Copyright © 1998 Rational Software, all rights reserved

A “4+1 View” Model of Architecture



Copyright © 1998 Rational Software, all rights reserved

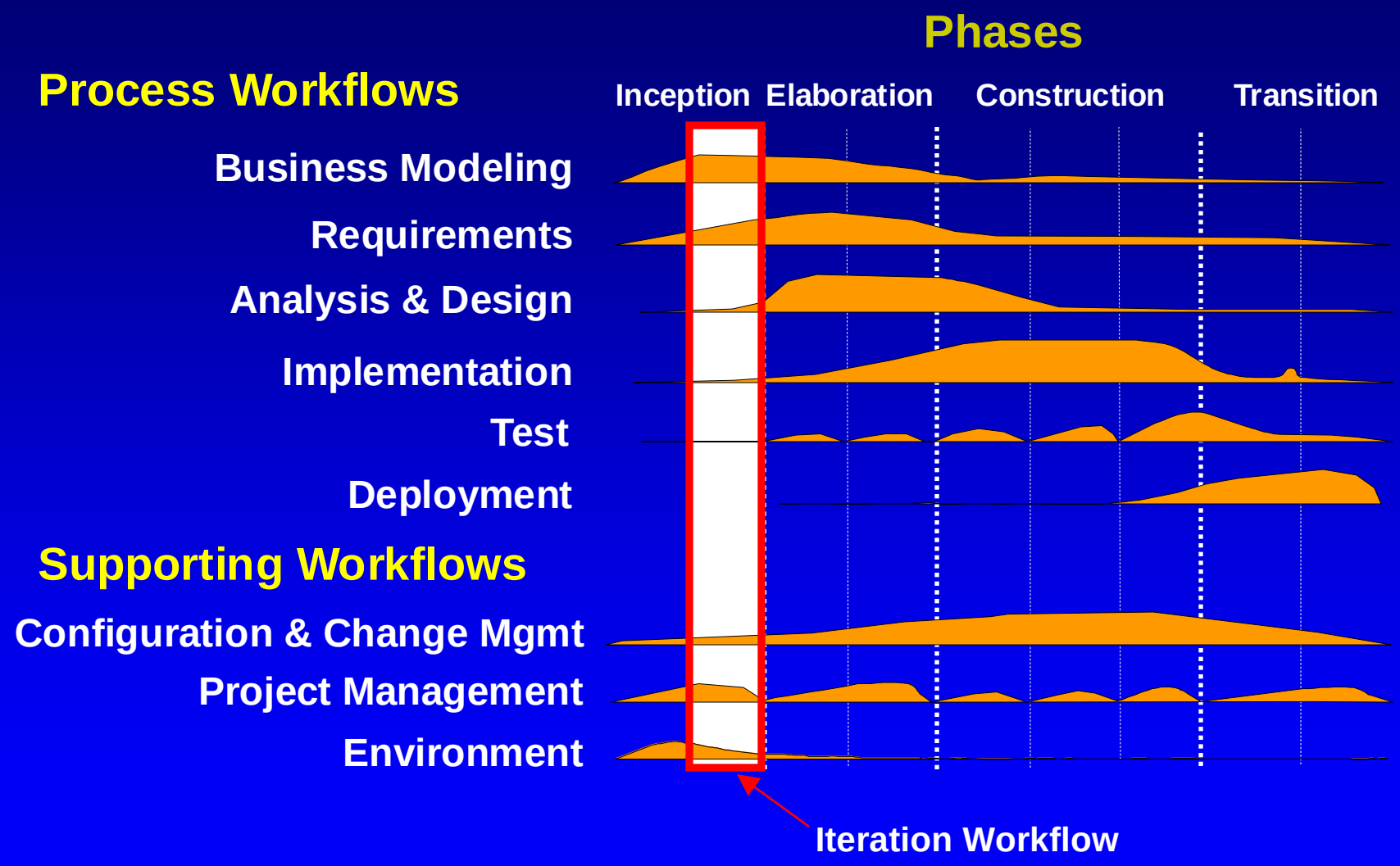
Iteration Workflows



Copyright © 1998 Rational Software, all rights reserved

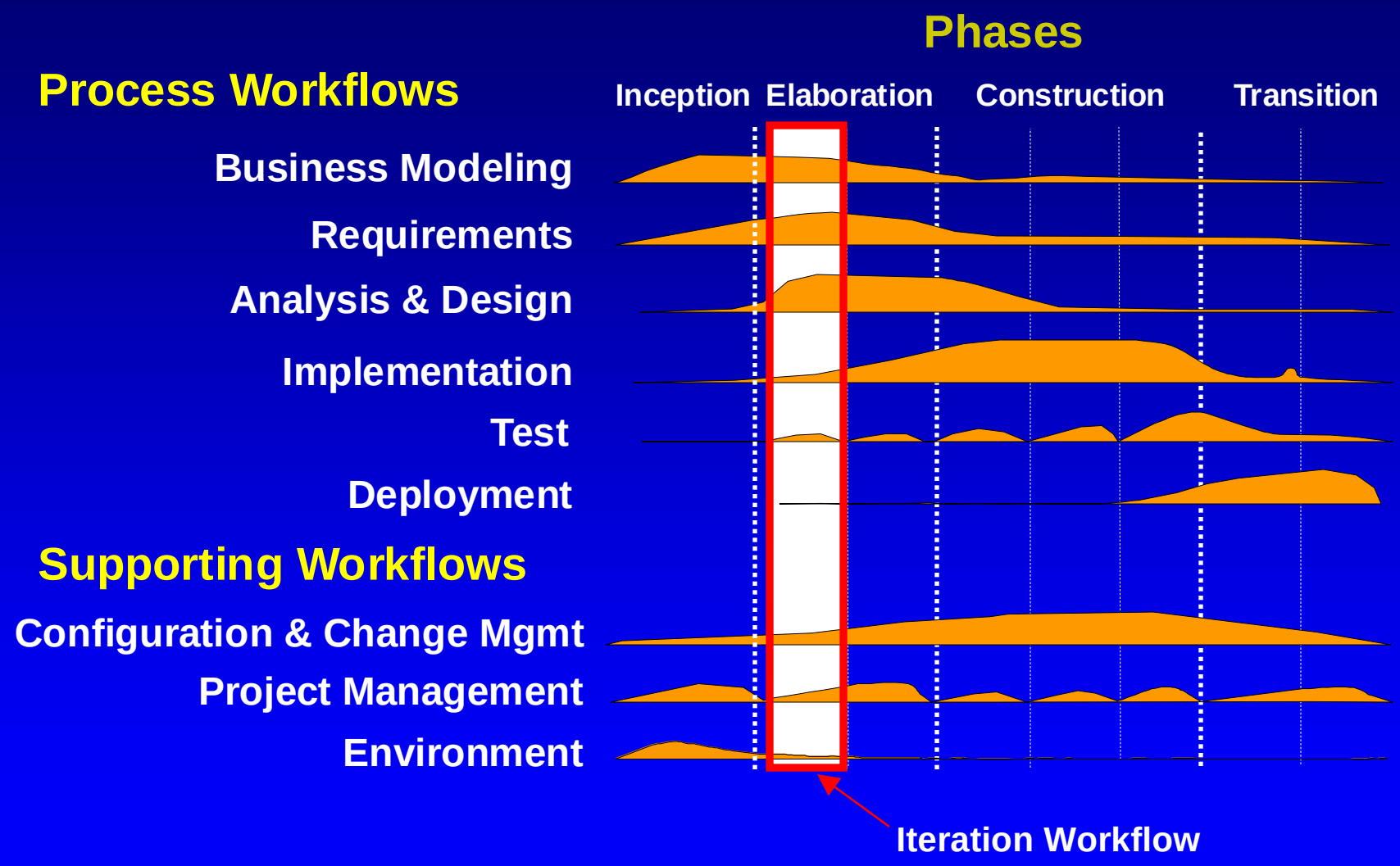
R

A Typical Inception Iteration



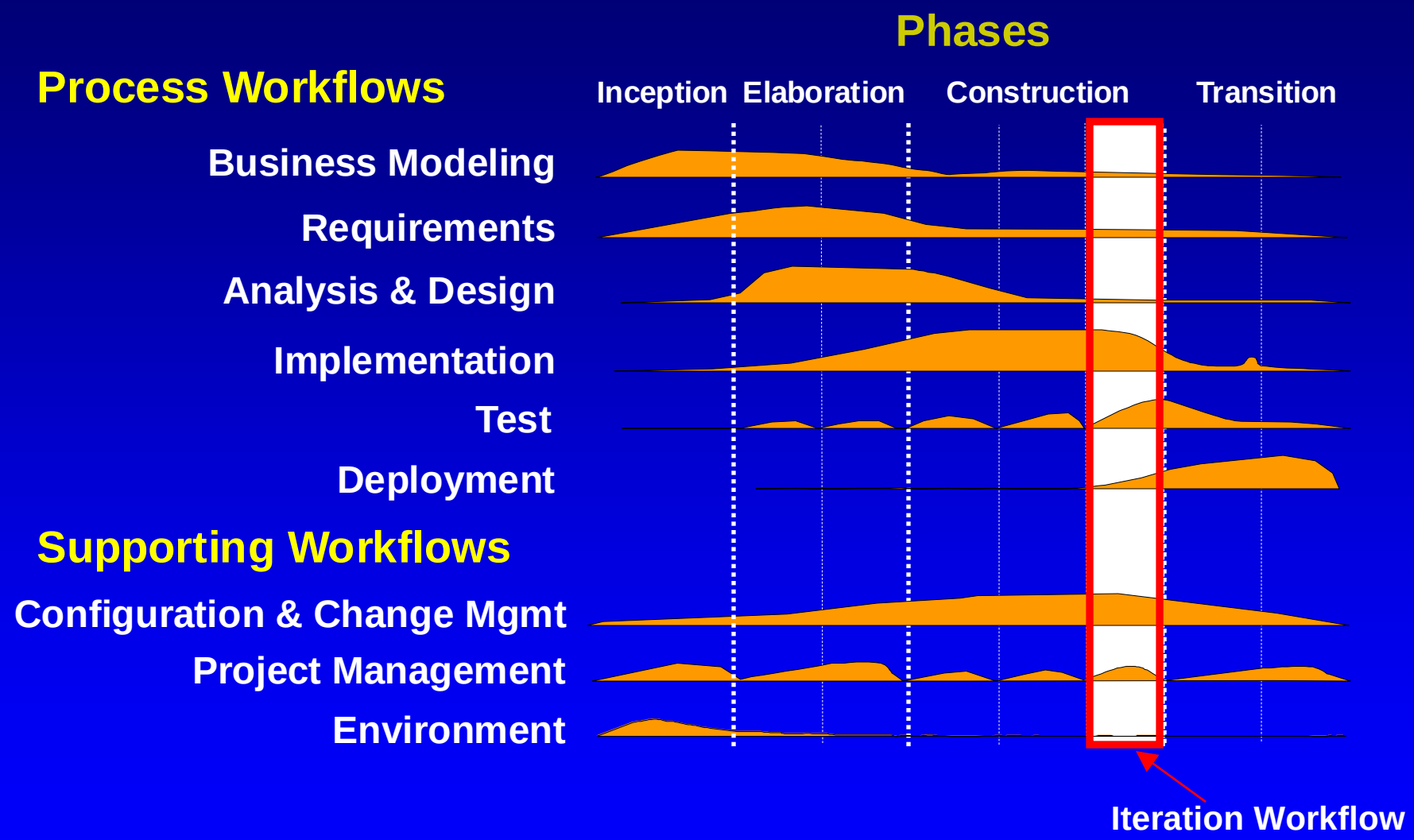
Copyright © 1998 Rational Software, all rights reserved

A Typical Elaboration Iteration



Copyright © 1998 Rational Software, all rights reserved

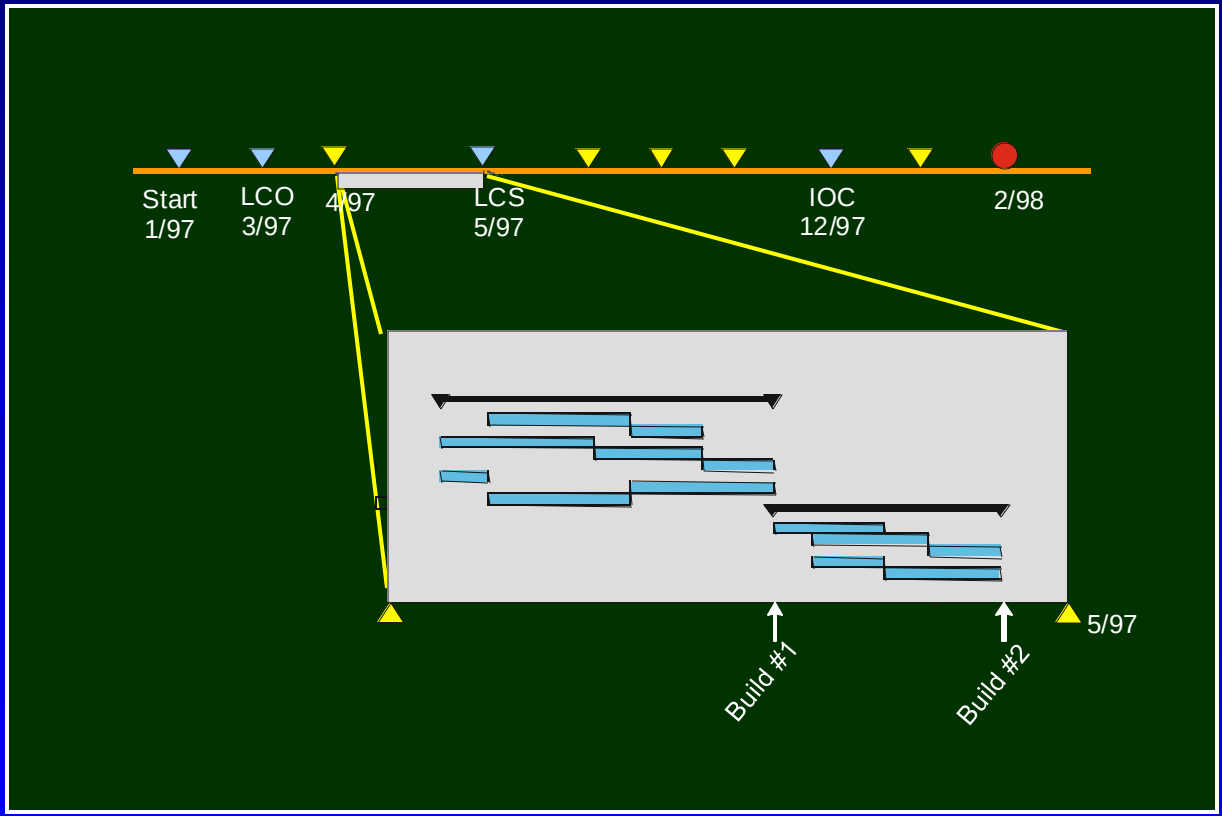
A Typical Construction Iteration



R

Copyright © 1998 Rational Software, all rights reserved

Project Management



Copyright © 1998 Rational Software, all rights reserved

R

Minor Milestones: Iteration Feedback Loop

Iteration N Cost and Schedule Actuals

Quality Assessment for Iteration N

- Test Results
- Defect Density
- Architecture Stability
- Other metrics

Iteration N Assessment

- Compare iteration actual cost, schedule, and content with iteration plan
- Determine rework (if any) to be done
  - Assign to future iteration(s)
- Determine what risks have been eliminated, reduced, or newly identified in this iteration
- Update project plan
- Prepare iteration plan for next iteration
  - Use revised risk list and select appropriate scenarios

Revised Risk List

Revised Project Plan

- Total Cost
- Overall Schedule
- Scope/Content

Iteration N+1 Plan

- Cost
- Schedule
- Content

R

Copyright © 1998 Rational Software, all rights reserved

Iteration Planning

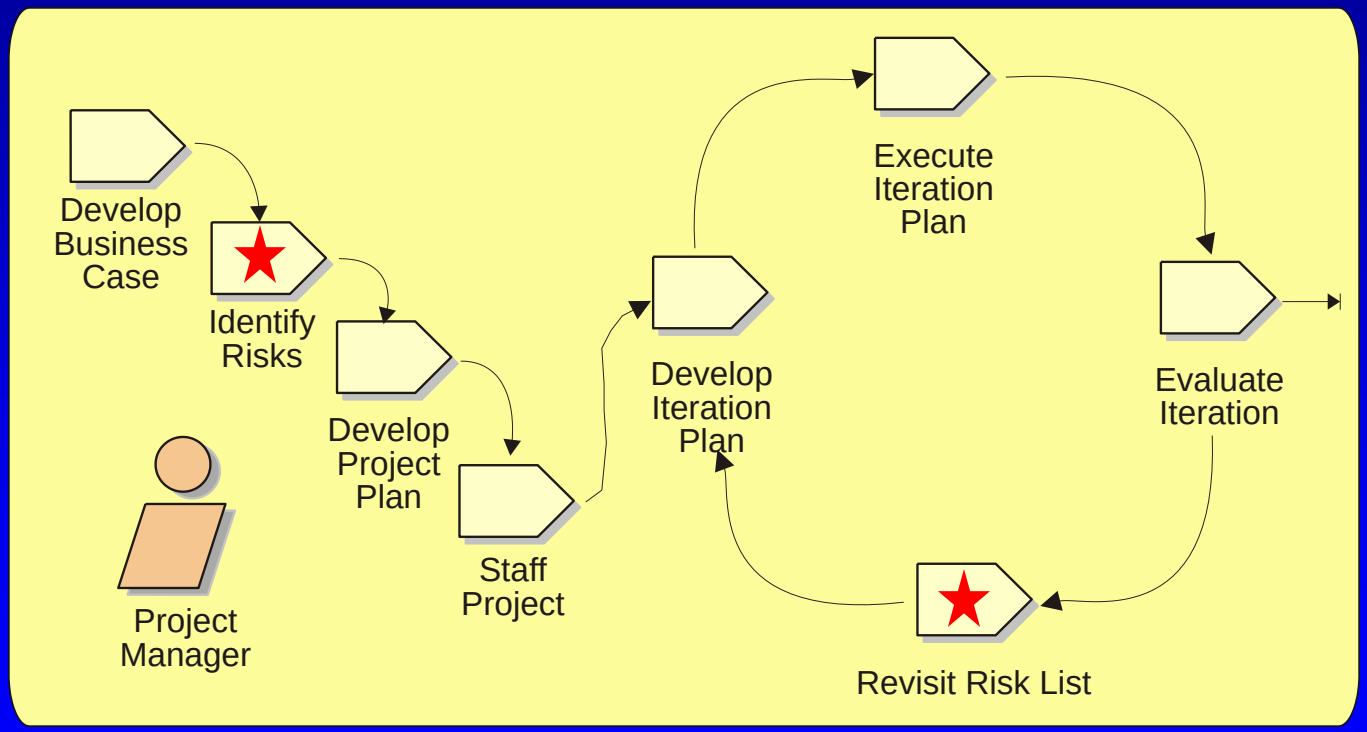
- Define objective evaluation criteria
- Identify what concrete, measurable artifacts will be developed or updated and the activities required to build them
- Use a standard work breakdown structure
- Use estimates to assign duration and effort to each activity
- Adjust as necessary to keep all numbers within resource constraints

Copyright © 1998 Rational Software, all rights reserved

R

Risks Drive the Iterative Lifecycle

- Risk assessment is a continuous process; risks change over time
- Early iterations address the highest risks



Copyright © 1998 Rational Software, all rights reserved

The Core Workflows

Requirements Capture:

From Vision to Requirements

Copyright © 1998 Rational Software, all rights reserved

R

List Candidate Requirements

- Create feature list
- For each feature
  - short name and description
  - status (proposed, approved, incorporated, validated)
  - estimated cost to implement (person-hours)
  - priority (must, should, could)
  - risk to implement (ordinary, significant, critical)
- Use list to allocate requirements to implementation iterations

Copyright © 1998 Rational Software, all rights reserved

R

Capture Functional Requirements

- Use cases can be used to capture functional requirements
  - represent one way of using the system
  - each user needs several use cases, typically
- Should also capture user interface which is exercised for each use case
  - sketch several versions
  - discuss with customer
- Do not get upset if customers change what you show them
  - better now than later

Copyright © 1998 Rational Software, all rights reserved

R

Resulting Artifacts

Work to be done	Resulting Artifacts
List candidate requirements	Feature list
Understand system context	Business or domain model
Capture functional requirements	Use-case model
Capture nonfunctional requirements	Supplementary requirements or individual use cases

Copyright © 1998 Rational Software, all rights reserved

R

Capturing the Requirements as Use Cases

Chapter 7

Copyright © 1998 Rational Software, all rights reserved

R

Introduction - Use Cases

- Develop model of system to be built
- Captures functional requirements
- Many nonfunctional requirements can be captured in use cases
  - for example, security requirements related to login
- Non-functional requirements unrelated to use cases captured in supplementary requirements
  - for example, overall system loading, or reliability
- Artifacts
- Workers
- Requirements capture workflow

R

Copyright © 1998 Rational Software, all rights reserved

Prioritize the Use Cases

- Determine which use cases need to be developed in early iterations
- Captures the architectural view of the use case model
- Project uses this view when planning the iterations

Copyright © 1998 Rational Software, all rights reserved

R

Analysis

Chapter 8

Refining and Structuring the Requirements

Copyright © 1998 Rational Software, all rights reserved

R

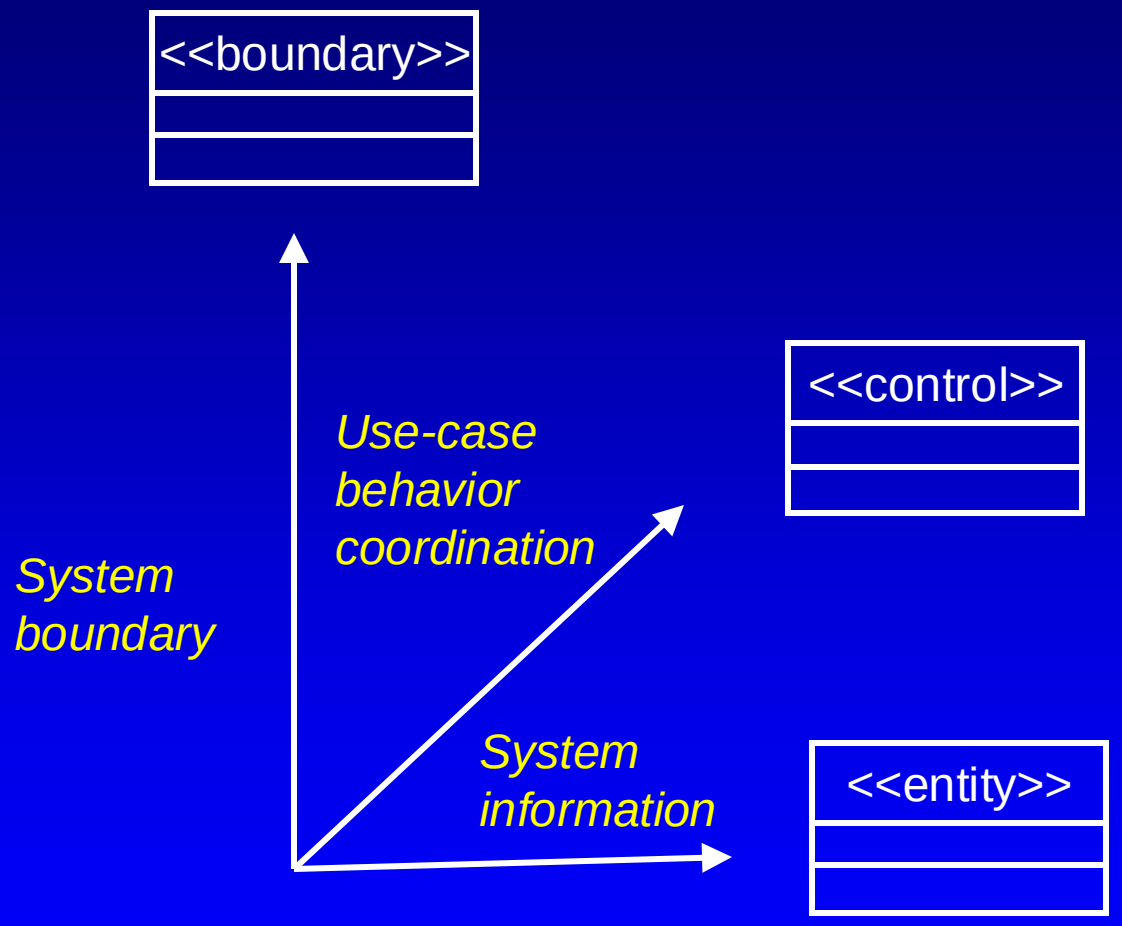
Introduction

- Purpose of analysis
  - more precise understanding of requirements
  - description or requirements that is easy to maintain
  - description of the requirements that gives structure to the system
- Move from language of customer to language of developer
- Start on internals of system
- Trace requirements from use-case model

Copyright © 1998 Rational Software, all rights reserved

R

What is an Analysis Class?

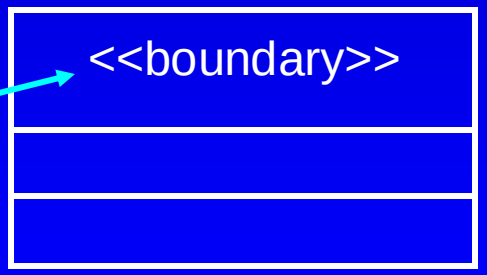


Copyright © 1998 Rational Software, all rights reserved

What is a Boundary Class?

- Intermediates the interface to something outside the system
- Several Types
  - User interface classes
  - System interface classes
  - Device interface classes
- *One boundary class per actor/use case pair*

Analysis class stereotype

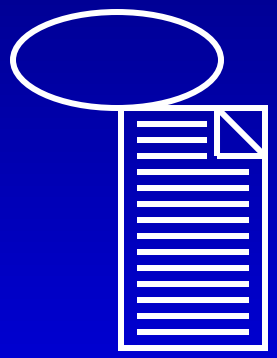


Environment Dependent

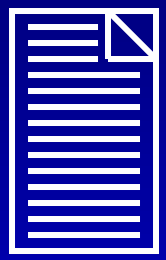
Copyright © 1998 Rational Software, all rights reserved

What is an Entity Class?

- Key abstractions of the system

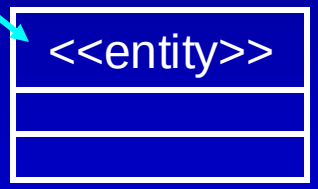


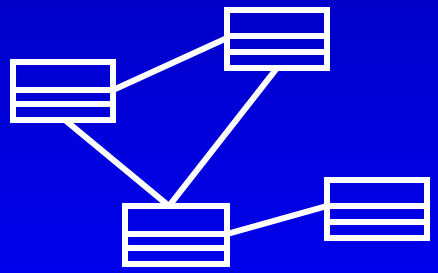
Use Case



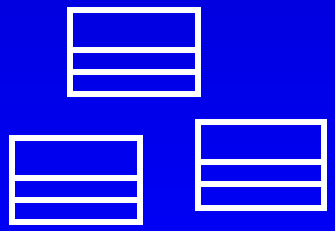
Glossary

Analysis class stereotype





Business-Domain Model



Architectural Analysis Abstractions

Environment Independent

Copyright © 1998 Rational Software, all rights reserved

R

What is a Control Class?

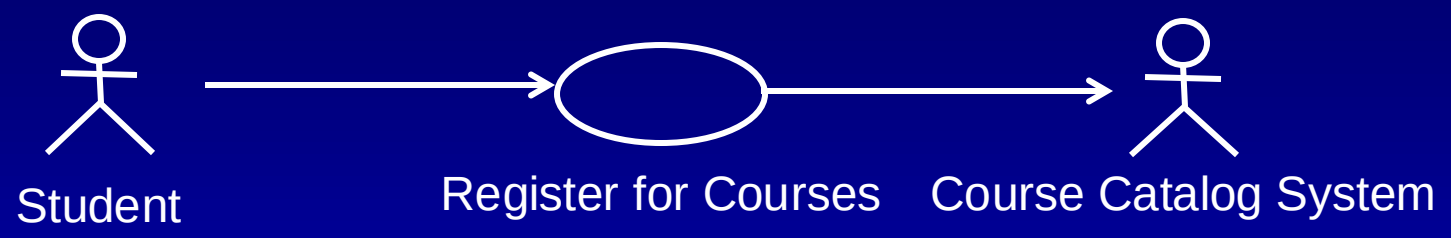
- Use-case behavior coordinator
- *One control class per use case*

Use-case dependent, Environment independent

Copyright © 1998 Rational Software, all rights reserved

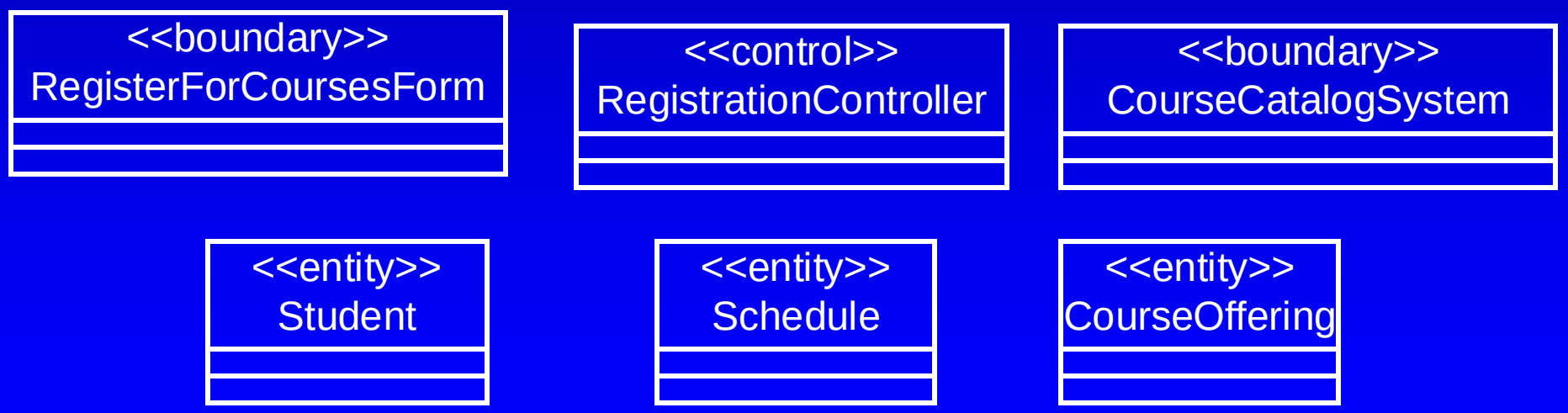
R

Example: Analysis Classes



Use-Case Model

Design Model

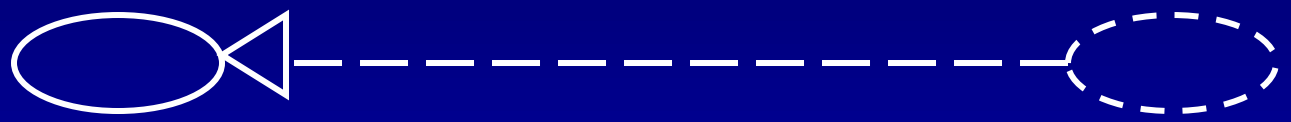


R

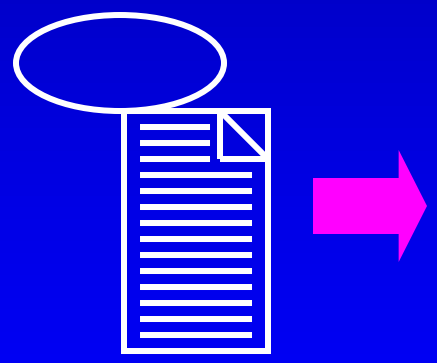
Copyright © 1998 Rational Software, all rights reserved

Use-Case Realization

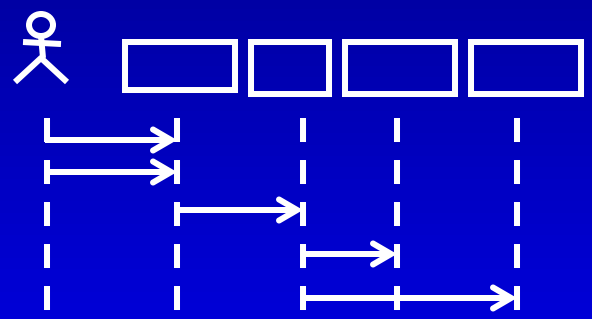
Use-Case Model *Design Model*



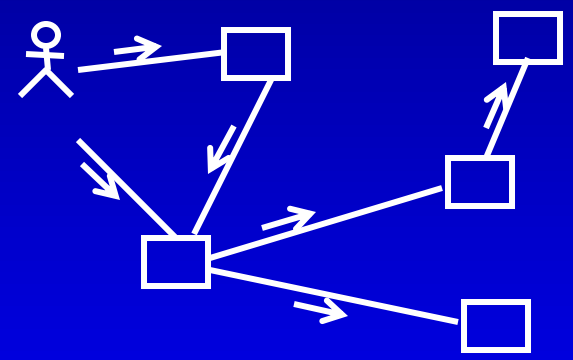
Use Case Use-Case Realization



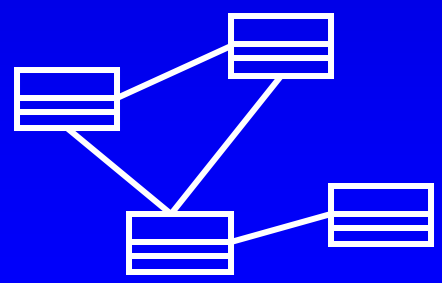
Use Case



Sequence Diagrams



Collaboration Diagrams



Class Diagrams

R

Copyright © 1998 Rational Software, all rights reserved

Design

Chapter 9

Copyright © 1998 Rational Software, all rights reserved

R

Purposes of Design

- Understand nonfunctional issues related to
  - programming languages
  - component reuse
  - operating systems
  - distribution and concurrency technologies
  - database technologies
  - user-interface technologies
  - transaction management technologies

Copyright © 1998 Rational Software, all rights reserved

R

Purposes of Design

- Create input for implementation
  - specify subsystems, interfaces, classes
- Decompose implementation into manageable pieces
  - often for implementation by different teams
- Capture and control major interfaces early
  - critical for gaining control over implementation
- Create seamless abstraction
  - structure preserved into code

Copyright © 1998 Rational Software, all rights reserved

R

Artifacts

- Design Model
- Design Class
- Use-Case realization - design
- Design subsystem
- Interface
- Architecture description (view of design model)
- Deployment model
- Architecture description (view of deployment model)

Copyright © 1998 Rational Software, all rights reserved

R

Use-Case Realization - Design

- Contains
  - class diagrams
    - can use to show participating classes
  - interaction diagrams
    - sequence diagrams show this graphically
  - flow of Events - Design
    - textual description
  - implementation requirements
    - textual description

Copyright © 1998 Rational Software, all rights reserved

R

Analysis vs. Design Model

- Conceptual model
- Design independent
- Analysis classes
- Less formal
- Less expensive
- Few layers
- Little focus on sequence
- Outlines design
- May not be maintained

- Physical model
- Implementation specific
- Physical stereotypes
- More formal
- 5:1 more expensive
- Many layers
- Much focus on sequence
- Manifests design
- Should be maintained

Copyright © 1998 Rational Software, all rights reserved

R

Actors, Agents and Servers

- According to Booch, objects can be categorized
  - whether they perform an active or passive role
- ACTOR: not acted upon by other objects
  - usually acts on other objects
- SERVER: is acted upon by other objects
  - doesn't act upon anything else
- AGENT: performs some operation on behalf of another object
  - may act upon other objects to do its job

Copyright © 1998 Rational Software, all rights reserved

R

Implementation

Chapter 10

Copyright © 1998 Rational Software, all rights reserved

R

Purpose of Implementation

- Plan system integrations required in each iteration
- Distribute system by mapping executable components onto nodes
- Implement design classes and subsystems
  - create source code
- Unit test components
- Integrate components into executables
  - feeds into test workflow

Copyright © 1998 Rational Software, all rights reserved

R

Integration Build Plan

- Describes sequence of builds required in an iteration
- For each build:
  - defines functionality expected to be implemented in the build
    - listing of use cases and/or scenarios or parts of them
    - can also refer to nonfunctional requirements
  - defines which parts of the implementation model are affected by the build
    - listing of subsystems and components to be implemented

Copyright © 1998 Rational Software, all rights reserved

R

Implement a Class

- At this point, it is a “simple matter of technique”
- Just code the thing and make sure it satisfies the design and provides the right interfaces
  - decide what to put in each source file (if needed)
  - there may be remaining decisions on how to implement associations and aggregations
  - algorithms may need to be chosen and designed
  - data structures may need to be designed
  - internal operations and data may be discovered

Copyright © 1998 Rational Software, all rights reserved

R

Perform Unit Test

- This workflow tests the implemented components as individual units
- The unit's externally observable behaviour should be tested
  - does it give proper output for all inputs?
  - does it handle error cases correctly?
- As well, the structure of the unit should be tested
  - has every line of code been tested, every branch?

Copyright © 1998 Rational Software, all rights reserved

R

Test

Chapter 11

Copyright © 1998 Rational Software, all rights reserved

R

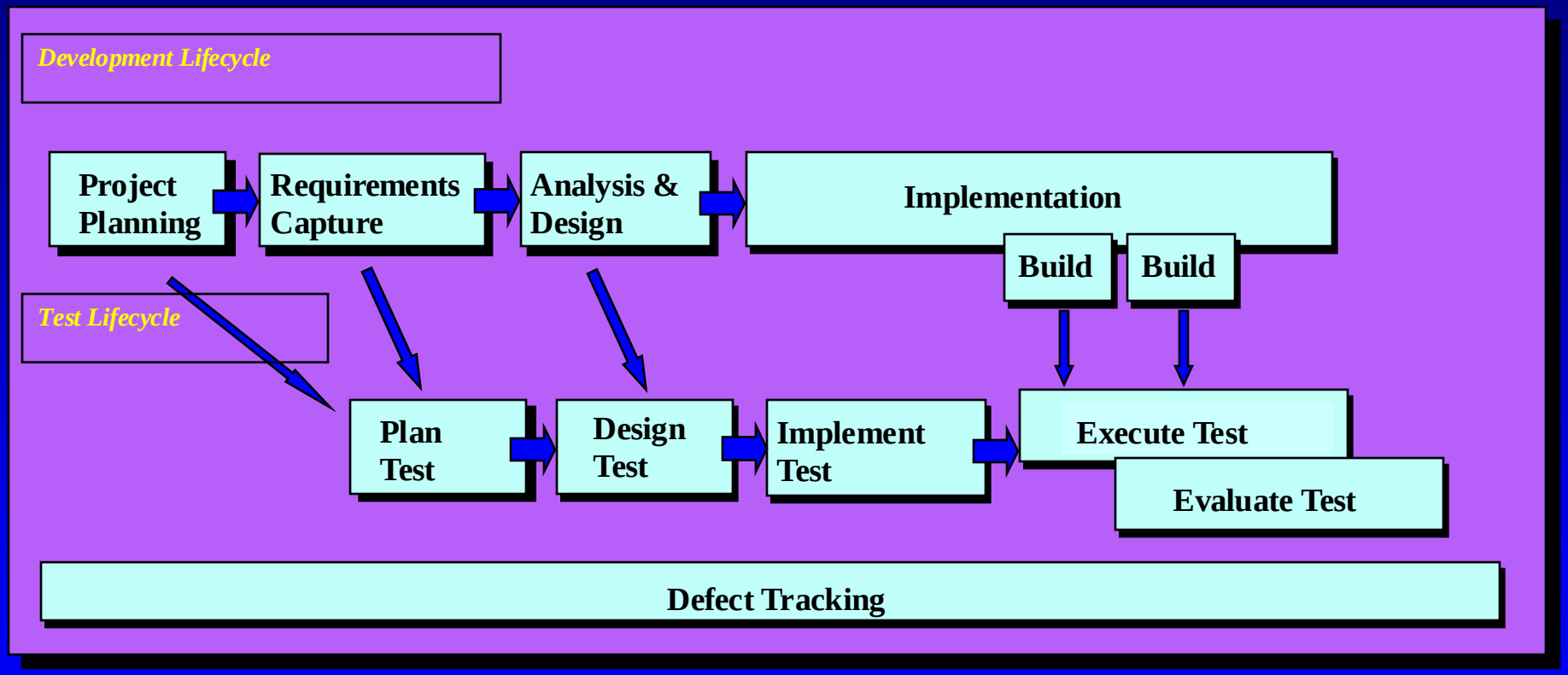
Testability

- How easy is it to test:
  - requirements, designs or code
- Major design decision:
  - how can this element be tested thoroughly
- Should not allow any non-verifiable requirements
  - some may have to be verified by other means than testing
- Testability doesn't just happen
  - have to consciously build it into a system

Copyright © 1998 Rational Software, all rights reserved

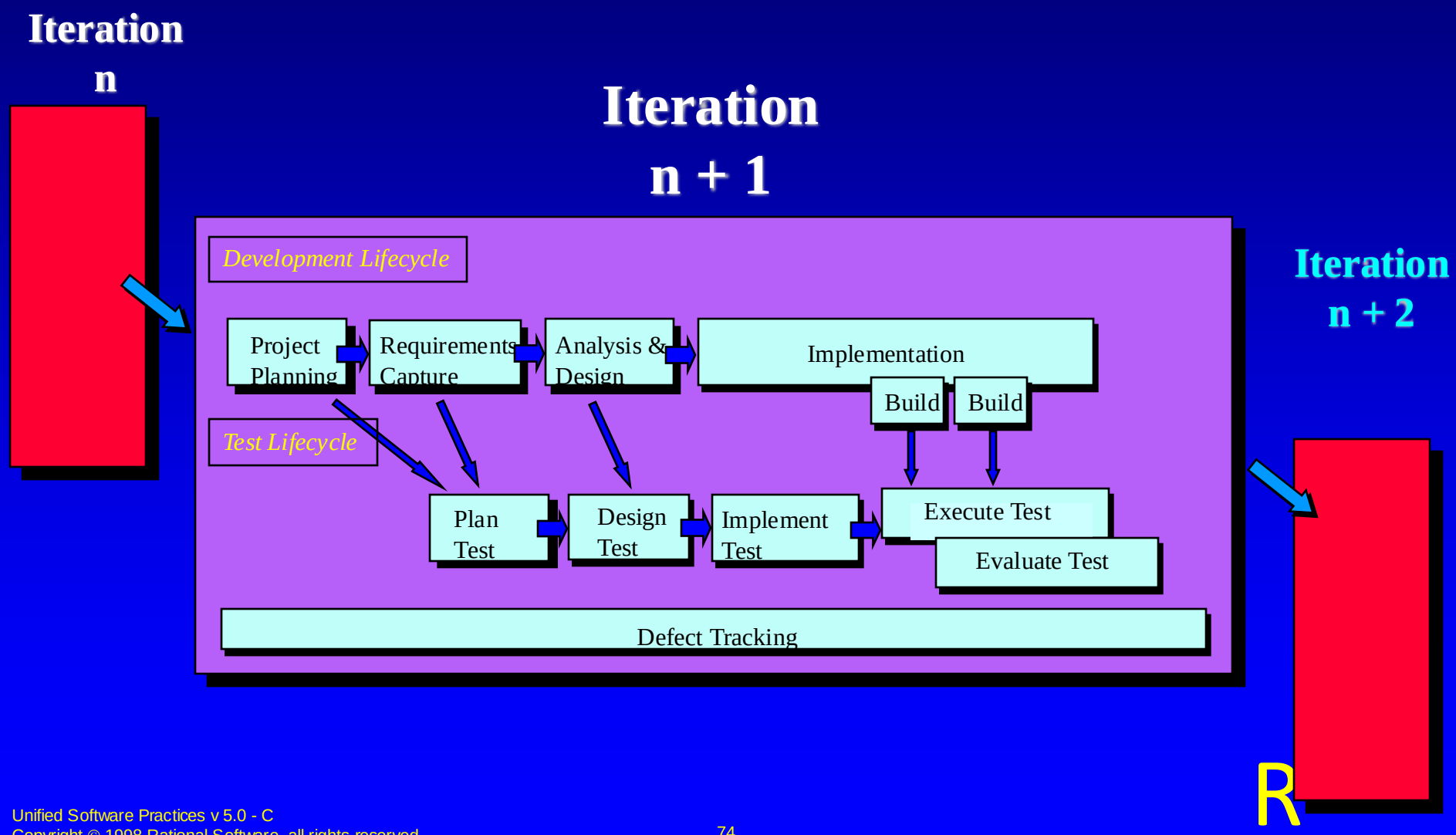
R

The Testing Lifecycle



Copyright © 1998 Rational Software, all rights reserved

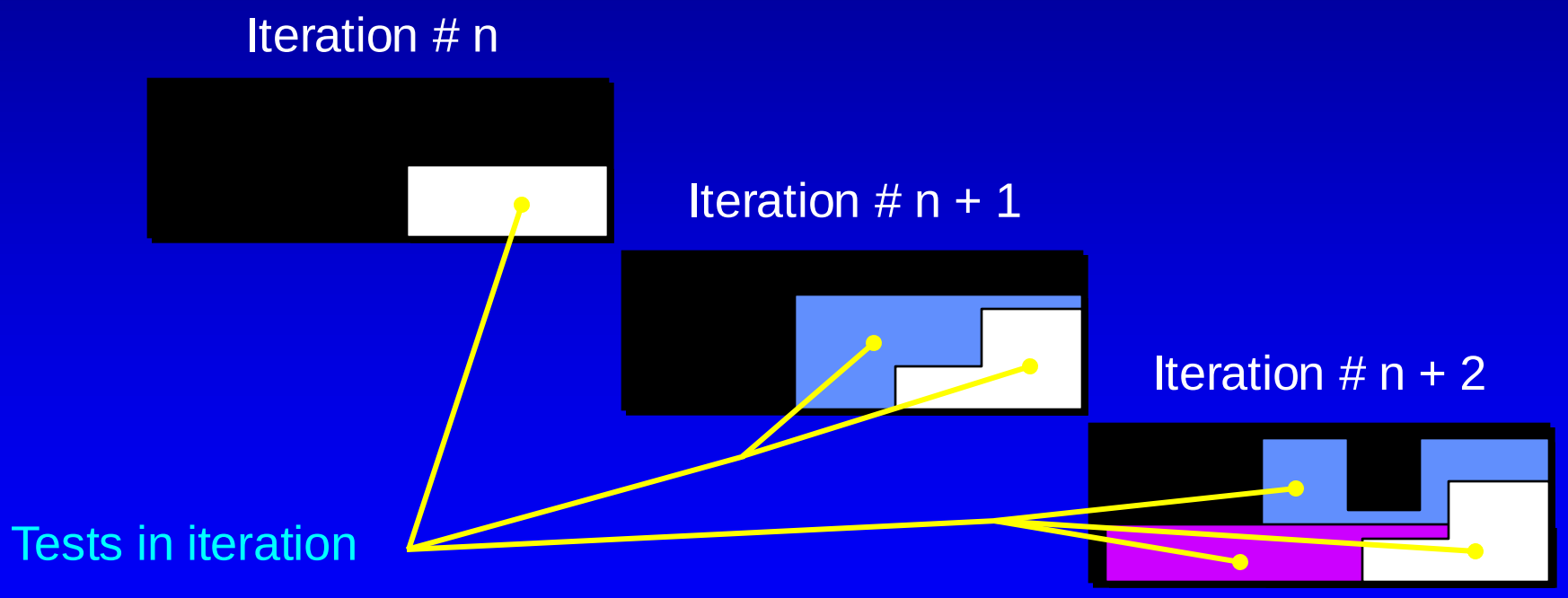
Iterations



Copyright © 1998 Rational Software, all rights reserved

Iterations

Tests evolve in iterations



Copyright © 1998 Rational Software, all rights reserved

Kinds of Testing

- Testing to verify different requirements:
  - Business function/rules
  - User interface
  - Input validation/syntax testing
  - Performance
  - Configuration/installation
  - Volume
  - Stress
  - Security
  - Recovery
  - Documentation

Copyright © 1998 Rational Software, all rights reserved

R

Generic Iteration Workflow

Chapter 12

Copyright © 1998 Rational Software, all rights reserved

R

What's Important?

- Early iterations
  - critical risks
  - key use cases
  - architectural issues
  - development environment
- Later iterations
  - development-oriented activities
  - implementation and testing
  - resolving performance issues

Copyright © 1998 Rational Software, all rights reserved

R

Risk Management

- Risks must be identified and then managed:
  - preventing the risk occurrence
    - selecting hardware that is available from multiple sources.
  - monitoring the risk
    - auditing the progress of contractors on whom you have a dependency.
  - taking corrective action
    - use a simulator until the hardware is ready, or hire consultants to make up for your staff shortfall.

Copyright © 1998 Rational Software, all rights reserved

R

Estimation Process Overview

- Estimate the size of the project (lines of code or function points)
- Estimate the effort (man-months)
- Estimate the schedule (calendar months)
- Provide estimates in ranges and refine the ranges as project progresses

Copyright © 1998 Rational Software, all rights reserved

R

Assess Iterations and Phases

- For iterative methods to work, need to assess what has been accomplished
  - at end of each iteration
  - at end of each phase
- Project Manager primarily responsible
- Two objectives
  - replan next iteration based on current results
  - modify process, tools, training, etc., based on experience

Copyright © 1998 Rational Software, all rights reserved

R

Disaster Recovery

R

Copyright © 1998 Rational Software, all rights reserved

Failing Projects

- People will not even guess at end dates
- Product is full of defects
- Extreme overtime, excess of 60 hours/week
- Management loses control, unclear on status
- Customer loses confidence in project delivery
- Team feels threatened about their progress
- Both sides considering cancellation
- Morale lousy, project is no fun anymore

Copyright © 1998 Rational Software, all rights reserved

R

General Recovery Options

- Cut the size of the software
- Increase process productivity
- Slip the schedule
- Drop a few features, increase productivity, slip schedule as needed

- Real problem is how to finish *at all*
  - focus on regaining control

Copyright © 1998 Rational Software, all rights reserved

R

The Plan

- Things to do first
- People issues
- Process issues
- Product issues
- Timing

Copyright © 1998 Rational Software, all rights reserved

R

Understanding Preferences

The effects of personality in the workplace

https://www.myersbriggs.org/

Copyright © 1998 Rational Software, all rights reserved

R

Primary Mental Activities

- When your mind is active, you use one of two mental activities:
  - taking in information, called *Perceiving*
  - organizing that information and coming to conclusions, called *Judging*
- Two ways of perceiving:
  - *Sensing* and *Intuition*
- Two ways of judging:
  - *Thinking* and *Feeling*

Copyright © 1998 Rational Software, all rights reserved

R

Focus of Attention

- People use the four functions in both the external world and in the internal world
  - Sensing
  - Intuition
  - Thinking
  - Feeling
- Using these in the external world is *Extraversion*, in the internal world is *Introversion*

Copyright © 1998 Rational Software, all rights reserved

R

Why are People Different?

- Jung believed everyone has a natural preference for using one kind of Perceiving and one kind of Judging
- He also observed that people are drawn more to the external world or to the internal world
- As people exercise these preferences, distinct perspectives and approaches develop to life and human interaction
- These lead to fundamental differences

Copyright © 1998 Rational Software, all rights reserved

R

Four Preference Scales

- Extraversion or Introversion
  - E 45.9/52.5%I 54.1/47.5%
- Sensing or Intuition
  - S 71.7/74.9%N 28.3/25.1%
- Thinking or Feeling
  - T 56.5/24.5% F 43.5/75.5%
- Judging or Perceiving
  - J 52.0/56.2% P 48.0/43.8%

Copyright © 1998 Rational Software, all rights reserved

R

Type Dynamics

- The four preferences indicate our choices from eight possibilities
- The middle two preferences (SN and TF) are called functions
- Type dynamics indicates the usual order of development of the four functions
- Usually, the more preferred are used more often and are better developed

Copyright © 1998 Rational Software, all rights reserved

R

Dominant Function

- The most preferred function is called the dominant function
- The dominant function takes the lead in shaping the personality
- People use the dominant function most in their favourite world
  - extraverts use dominant more in external world
  - introverts use dominant more in internal world
    - this makes the dominant of introverts harder to see

Copyright © 1998 Rational Software, all rights reserved

R

Auxiliary Function

- The other function in the type code is called the auxiliary function (other of middle letters)
- Everyone needs both information and decision-making
- If dominant is an information-gathering function (SN), then auxiliary will be a deciding function (TF) and vice versa
- Auxiliary is more often used in less favoured world

Copyright © 1998 Rational Software, all rights reserved

R